

基于多平台协同与多算法优化的烟幕干扰弹投放策略研究

摘要

烟幕干扰弹利用化学燃烧或爆炸形成烟幕，有效干扰导弹识别与攻击。本文针对无人机投放过程中的路径规划、投放与起爆控制，构建无人机—导弹—烟幕云团的时空协同优化模型，并采用多目标优化与轨迹预测方法研究不同任务条件下的飞行与投放策略。

针对问题一：构建导弹与烟幕云团的三维相对运动模型与完全遮蔽几何判断模型，计算 FY1 发射 1 枚烟幕干扰弹对导弹 M1 的有效遮蔽时长。题目已知位置与速度，利用动力学知识，建立导弹、无人机和烟幕干扰弹三维运动学模型；继而利用完全遮蔽的定义，通过几何推导得到烟幕遮蔽几何判断模型。联立求解上述模型，得到有效遮蔽时长为 **1.4050 s**，同时可得到具体遮蔽时间区间为 **[8.043187, 9.448222]s**。

针对问题二：根据单导弹·单无人机·单烟幕弹的情形，建立以最大化完全遮蔽时长为目标的[1 1 1]优化模型，将无人机航向角 θ_1 、飞行速度 v_{U1} 、投放时间以及引信延迟作为决策变量，约束条件包括角度大小、速度范围，时间限制，起爆高度等。基于问题一的三维运动学模型与几何遮蔽判定模型，采用退火改进粒子群组合算法（PSO-SA）进行全局搜索与局部精细化求解。最优飞行投放策略为 $\theta_1 = 7.19^\circ$ ， $v_{U1} = 131.89m/s$ ，投放点 **(17800, 0, 1800)m**，起爆点 **(17897, 12, 1797)m**，最大遮蔽时长为**4.590 s**。

针对问题三：根据单导弹·单无人机·多烟幕弹的情形，建立[1 1 3]最大遮蔽时间优化模型，沿用三维相对运动与几何遮蔽判定模型，同时在问题二的基础上增加投放间隔的时间约束。采用“差分进化（DE）全局搜索+粒子群（PSO）局部抛光”的优化思想。得到无人机投放决策策略(详见 **result2.xlsx**)，最长有效遮蔽时长为 **6.403159 s**。通过收敛性验证，该组合优化算法稳定收敛且精度高。

针对问题四：根据单导弹·多无人机·单烟幕弹的情形，建立[1 3 1]最大遮蔽时间优化模型，最大化总有效遮蔽时长。利用子问题二，将三机策略分解为单机子问题独立优化，并验证遮蔽区间互不重叠，可用贪心合成总时长，得到各无人机最优投放与起爆参数(详见 **result2.xlsx**)及最长有效遮蔽总时间为**11.4917s**。验证采用“粗扫描+高精度二分法”局部精细化搜索，得到有效遮蔽为**11.100s**，表明算法高效稳定、结果准确。

针对问题五：根据多导弹·多无人机·多烟幕弹的复杂协同防御任务，提出融合任务分配与时空协同优化的全局框架，最大化系统总有效遮蔽时间。我们创新性地引入分解-组合优化策略，将全局问题拆解为“单无人机·多导弹”局部最优子问题，通过差分进化（DE）、粒子群优化（PSO）与模拟退火（SA）混合求解，实现兼顾全局搜索能力与局部精修性能；同时通过遮蔽区间的并集处理与时间重叠惩罚机制，得到时空资源利用率最优的全局解，无人机投放决策策略参数详见 **result3.xlsx**，有效总遮蔽为**22.581s**。

关键字：多平台协同；多算法优化；烟幕干扰弹；无人机投放策略；三维运动学模型

一、 问题重述

1.1 问题背景

烟幕干扰弹作为一种高效低成本的新型防御手段，其技术设计与战术运用具有较强的系统性。精准的时空协同与多单元控制能力，是烟幕干扰系统发挥效能的关键。一套完整的烟幕干扰任务通常由多架无人机、多枚干扰弹协同完成，覆盖空域广阔。执行任务时，无人机需快速调整航向与速度，在导弹来袭路径前方精确投放干扰弹，通过时序起爆形成具有一定持续时间与空间分布的烟幕区域。在此期间，应力求实现对导弹视线的有效遮蔽，遮蔽总时间越长，则整体干扰效果越佳。

由于导弹速度极高，加之烟幕云团具有下沉与时效限制，往往难以实现完全连续的理想遮蔽。因此，为提升烟幕干扰的整体防御能力，需重点研究无人机的飞行路径规划、干扰弹的投放策略与起爆控制等关键参数，以延长有效遮蔽时间。

1.2 问题提出

基于上述背景，要求建立数学模型解决下列问题。

一枚烟幕干扰弹形成有效遮蔽的范围为以爆点为中心、半径 10m 的球形区域，烟幕云团以 3m/s 匀速下沉，有效遮蔽时间为起爆后 20s 内。每架无人机可在 70~140m/s 范围内选择飞行速度，并可在受领任务后瞬时调整航向，之后保持等高度匀速直线飞行。投放多枚干扰弹时，同一无人机两次投放间隔不少于 1s。假目标为半径 7 m、高 10 m 的圆柱，假目标为坐标原点中心，真目标下底面的圆心为 (0, 200, 0)。

来袭导弹速度为 300m/s，飞行方向指向假目标点。初始时刻，三枚导弹 M1、M2、M3 分别位于(20000, 0, 2000)、(19000, 600, 2100)、(18000, -600, 1900)；五架无人机 FY1~FY5 的初始位置分别为(17800, 0, 1800)、(12000, 1400, 1400)、(6000, -3000, 700)、(11000, 2000, 1800)、(13000, -2000, 1300)。

问题一：利用无人机 FY1 以 120m/s 的速度朝向假目标方向飞行，受领任务 1.5s 后投放 1 枚烟幕干扰弹，间隔 3.6s 起爆。据此计算烟幕干扰弹对导弹 M1 的有效遮蔽时长。

问题二：仍利用无人机 FY1 投放 1 枚烟幕干扰弹实施对 M1 的干扰，需确定 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点及起爆点，使得烟幕干扰弹对 M1 的有效遮蔽时间尽可能长。

问题三：利用无人机 FY1 投放 3 枚烟幕干扰弹实施对 M1 的干扰。制定烟幕干扰弹的投放策略，包括各弹的投放时刻、起爆时刻及 FY1 的飞行参数，使得烟幕干扰弹对 M1 的有效遮蔽时间尽可能长。

问题四：利用 FY1、FY2、FY3 三架无人机各投放 1 枚烟幕干扰弹，协同实施对 M1

的干扰。制定多无人机协同的烟幕干扰弹投放策略，使得烟幕干扰弹对 M1 的有效遮蔽时间尽可能长。

问题五：利用全部 5 架无人机（FY1 至 FY5），每架无人机至多投放 3 枚烟幕干扰弹，对 M1、M2、M3 三枚导弹实施协同干扰。制定整体投放策略，实现对 M1、M2、M3 的有效遮蔽总时长最大化。

二、问题分析

2.1 问题一

问题一的核心在于已知无人机 FY1 固定速度与方向条件下，对导弹 M1 投放一枚烟幕干扰弹所能产生的有效遮蔽时长。导弹速度极高，轨迹固定；烟幕云团下沉速度较慢且有效半径有限，由此可知遮蔽时间受云团起爆时间，以及导弹与真实目标的相对位置的限制。无人机、导弹初始位置与速度、烟幕投放时间与起爆延迟均已知，云团下沉速度固定。我们对有效遮蔽的定义为云团完全阻断导弹到真目标的视线。因此，我们计划通过三维运动学分析导弹与烟幕的相对位置变化，结合对目标圆柱体进行表面采样判定几何视线遮挡，确定烟幕有效遮蔽的时间区间，从而累加得到有效遮蔽时长。

2.2 问题二

问题二在于可调整飞行方向和速度的条件下，确定无人机 FY1 的最优飞行参数及投放、起爆点，使对 M1 的遮蔽时间最大化。无人机可在任务开始瞬时改变航向与速度，并自行决策烟幕弹的投放时刻以及引信延迟，然而导弹速度远大于无人机，因此我们分析最佳投放位置非常受限。对此我们只有使烟幕与导弹—真目标的连线区域在时间和空间上高度匹配，才能在关键时刻形成遮蔽。因此，我们建立无人机与导弹的相对运动模型，将遮蔽时间作为目标函数，在速度范围、投放间隔和引信延迟等约束条件下，通过全局优化算法探索最优参数组合，得到最佳飞行策略。

2.3 问题三

问题三要求设计 FY1 投放三枚烟幕干扰弹对 M1 实施干扰的策略，使总遮蔽时间尽可能长。多枚烟幕可在时间轴上形成多段遮蔽区间理论上总遮蔽有效时间可以有效增加。问题的关键在于若投放间隔或位置选择不当，可能导致遮蔽区间重叠，浪费有效覆盖。因此，我们通过分析时间上导弹与各个烟幕的相对运动，得到最佳投放序列。

2.4 问题四

问题四提出三架无人机（FY1~FY3）各投放一枚烟幕，对 M1 实现协同遮蔽，目标是最大化总遮蔽时间。各无人机初始位置不同，可独立选择航向、速度、投放参数。多

平台协同可增加空间覆盖，关键在于空间时序调度的策略，避免彼此干扰。我们需要在全局上协调避免烟幕弹遮蔽时间重叠，从而使得优化目标达到最优解。

2.5 问题五

问题五的关键在于五架无人机对三枚导弹同时干扰，每架最多投放三枚烟幕，需在全局上最大化系统有效遮蔽时间。此时，问题的核心不再是某一枚导弹的最优策略，而是系统整体的最小遮蔽时间最大化。由于导弹数量多、速度快、初始位置分布不同，且无人机位置与投放次数均受限，必须在导弹之间合理分配烟幕资源。

全局难度在于，不同导弹的最佳方案可能相互冲突，单一目标的优化会削弱整体防护效果。因此，应采用混合整数非线性规划方法，将任务分配、飞行参数与投放时机的优化统一建模，并结合全局搜索与局部精细化策略，权衡资源分配，并得到系统整体最佳遮蔽效果。

三、模型假设

● 运动学假设：

1. **导弹轨道固定且视为质点：**导弹在飞行过程中匀速直线飞向假目标 O ，途中不受烟幕弹的影响而改变飞行角度；并且导弹在空中运动时视为质点。
2. **无人机受领任务后瞬间调整角度与速度：**无人机受领任务后瞬时调整方向，然后固定速度和方向匀速直线飞行。
3. **烟幕弹运行轨迹：**烟幕弹投放后忽略空气阻力，在重力作用下做平抛运动；起爆后瞬时形成球体，由于烟雾所受空气阻力影响较大，以及采用特定技术，该烟幕云团以 3 m/s 的速度匀速下沉。

● 情景假设：

1. **不考虑烟幕云团的空间交叠：**烟幕云团(半径 10 m 的球形)在三维空间交叠，可能造成资源浪费。但是由于优化目标中包含的待决策参数较多，涉及的计算量较大，因此我们在这里假设不考虑烟幕云团在空间上的资源浪费。
2. **有效遮蔽定义：**烟幕弹起爆后形成的云团完全阻挡导弹对真目标的视线，即导弹无法观测到目标的任何表面店，从而保护真目标免被导弹识别定位。

3. 多烟幕/多导弹情形：

单导弹多烟幕：同一导弹的有效遮蔽时间可不连续，不同烟幕弹可叠加遮蔽效果。同一时刻若被多个烟幕遮蔽，总遮蔽时间按并集计算。

多导弹评估：以最短遮蔽时间的导弹为准评估威胁，该导弹对目标危险性最高。

四、符号说明

符号	说明	单位	已知值/范围
P_{target}	假目标坐标	m	$(0, 0, 0)$
$M_m(t)$ 、 $U_i(t)$	第 m 枚导弹坐标、第 i 架无人机坐标	m	时间函数
v_M 、 v_{U_i}	导弹、无人机飞行速度	m/s	$300m/s$ 、决策变量
u_{M_m}	导弹飞行方向单位向量	m/s	u_{M_m} 大小方向已知
u_{U_i}	第 i 架无人机飞行方向单位向量	m/s	u_{U_i} 方向为决策变量
θ_{U_i}	无人机飞行角度(x 轴逆时针)	$^\circ$ (度)	决策变量
$t_{rel}^{(i,j)}$	第 i 架无人机第 j 枚干扰弹的投放时间	s	决策变量
$\tau_{fuse}^{(i,j)}$	引信延迟	s	决策变量
$P_{drop}^{(i,j)}$	第 i 架无人机投放第 j 枚干扰弹的位置	m	运动方程求解
$P_{explode}^{(i,j)}$	第 i 架无人机投放第 j 枚干扰弹起爆位置	m	运动方程求解
T_{cover}^m 、 $T_{effective}$	第 m 枚导弹、总系统有效遮蔽时间	s	优化目标

表 1、符号说明

五、模型建立与求解

5.1 问题一模型的建立与求解

为了计算无人机 FY1 投放烟幕干扰弹对来袭导弹 M1 的有效遮蔽时长，我们建立了导弹与烟幕云团的三维相对运动模型，并利用线段-球体相交判定是否遮蔽，计算导弹在目标各采样点被球状烟幕完全遮蔽的时间区间，从而求得有效遮蔽时长。

5.1.1 问题一：导弹与烟幕云团的三维相对运动模型的建立

● 导弹运动模型的建立

导弹 M_1 从初始位置 $(20000,0,2000)$ 以 $300m/s$ 速度沿直线飞向假目标，已知假目标设为坐标原点 O 。则我们可以求得导弹 M_1 运动的单位方向向量为：

$$u_{M_1} = \frac{O - M_1(0)}{|O - M_1(0)|} = \left(-\frac{20000}{\sqrt{20000^2 + 2000^2}}, 0, -\frac{2000}{\sqrt{20000^2 + 2000^2}} \right) = (-0.995, 0, 0.0995)$$

导弹位置随时间 t 的变化为：

$$M_1(t) = M_1(0) + v_{M_1} \cdot t \cdot u_{M_1}$$

- 无人机运动模型的建立

无人机 FY1 从(17800,0,1800)出发, 以 $v_{U_i} = 120\text{m/s}$ 的速度朝假目标方向等高度匀速飞行, 同理可求得无人机运动的单位方向向量:

$$u_{U_i} = (\cos\theta_i, \sin\theta_i, 0)$$

θ_{U_i} 为无人机飞行角度(x 轴逆时针), 该题已知无人机 FY1 朝假目标方向匀速飞行, 因此 θ_{U_i} 已知。

无人机的位置随时间 t 的变化为:

$$U_i(t) = U_i(0) + v_{U_i} \cdot t \cdot u_{U_i}$$

则可得, 无人机FY1第1枚干扰弹的投放位置为:

$$P_{drop}^{(1,1)} = U_1(t_{rel}^{(1,1)}) = U_1(0) + v_{U_1} \cdot t_{rel}^{(1,1)} \cdot u_{U_1}$$

- 烟幕弹运动模型的建立:

根据题干给出的信息, 任务受领 1.5s 后投放烟幕干扰弹, 烟幕弹在 3.6s 后起爆, 烟幕弹的起爆点由无人机轨迹和烟幕弹下落运动决定, u_{U_1} 为无人机FY1飞向假目标方向单位向量, k 为竖直向下单位向量, t_{rel} 为任务受领延迟, t_{fuse} 为烟幕弹引信延迟, k 为竖直向下单位向量, g 取 9.81m/s :

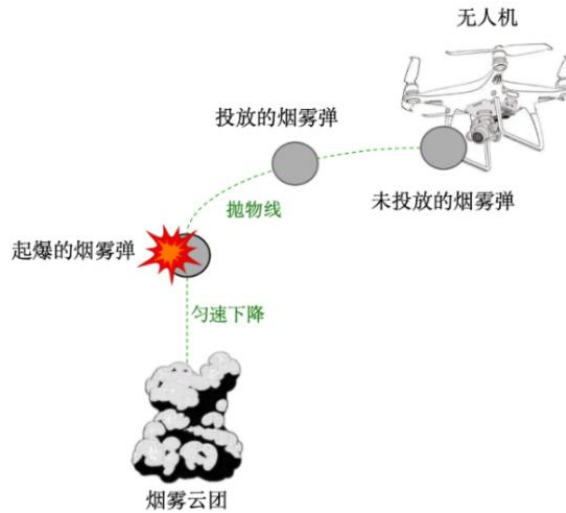


图 1. 烟幕弹投掷、起爆运动过程示意图

无人机FY1第1枚干扰弹在起爆前做平抛运动, 则它的起爆位置为:

$$P_{explode}^{(i,j)} = P_{U_1}(t_{fuse}^{(i,j)}) = \begin{cases} x_e = P_{drop,x}^{(i,j)} + v_{horiz,x}^{(i,j)} \cdot \tau_{fuse}^{(i,j)} \\ y_e = P_{drop,y}^{(i,j)} + v_{horiz,y}^{(i,j)} \cdot \tau_{fuse}^{(i,j)} \\ z_e = P_{drop,z}^{(i,j)} + \frac{1}{2} \cdot g \cdot (\tau_{fuse}^{(i,j)})^2 \cdot k \end{cases}$$

可得问题 1 中烟幕弹起爆弹位置为：

$$P_{explode}^{(1,1)} = P_{drop}^{(1,1)} + v_{U_1} \cdot \tau_{fuse}^{(1,1)} \cdot u_{U_1} + \frac{1}{2} \cdot g \cdot (\tau_{fuse}^{(1,1)})^2 \cdot k$$

同时易得烟幕弹起爆时刻 $t_{explode}^{(1,1)}$ 为：

$$t_{explode}^{(1,1)} = t_{rel}^{(1,1)} + \tau_{fuse}^{(1,1)} = 1.5s + 3.6s = 5.1s$$

烟幕弹起爆后，形成半径 $R = 10m$ 的球状烟幕云团，云团以 $v_s = 3m/s$ (v_s 方向竖直向下)匀速下沉，并在起爆20s内保持有效遮蔽。

则可得烟幕弹的有效时间窗为 $[5.100, 25.100] s$ 。

云团中心的运动坐标表示如下：

$$P_{Center}^{(1,1)}(t) = P_{explode}^{(1,1)} - v_s \cdot (t - t_{explode}^{(1,1)}), \quad t \in [t_e, t_e + 20]$$

- 导弹与烟幕云团的三维相对运动模型联立

$$\begin{cases} M_1(t) = M_1(0) + v_{M_1} \cdot t \cdot u_{M_1} \\ P_{explode}^{(1,1)} = P_{drop}^{(1,1)} + v_{U_1} \cdot \tau_{fuse}^{(1,1)} \cdot u_{U_1} + \frac{1}{2} \cdot g \cdot (\tau_{fuse}^{(1,1)})^2 \cdot k \\ P_{Center}^{(1,1)}(t) = P_{explode}^{(1,1)} - v_s \cdot (t - t_{explode}^{(1,1)}), \quad t \in [t_{explode}^{(1,1)}, t_{explode}^{(1,1)} + 20] \end{cases}$$

$$t_{explode}^{(1,1)} = t_{rel}^{(1,1)} + \tau_{fuse}^{(1,1)} = 1.5s + 3.6s = 5.1s$$

5.1.2 问题一：导弹与烟幕云团的三维相对运动模型的求解

- 完全遮蔽几何判断模型

真目标被建模为圆柱体，高度 $H = 10m$ ，半径 $R_{cyl} = 7m$ 。通过对几何图形的直观理解，我们发现应该重点关注目标边界是否被导弹检测到，因此我们在顶面圆和底面圆的边界随机采样若干点 P_i ，若从导弹到任意顶面圆和底面圆的边界随机采样点的线段与云团球体相交，即该时间点真目标被烟幕遮蔽。

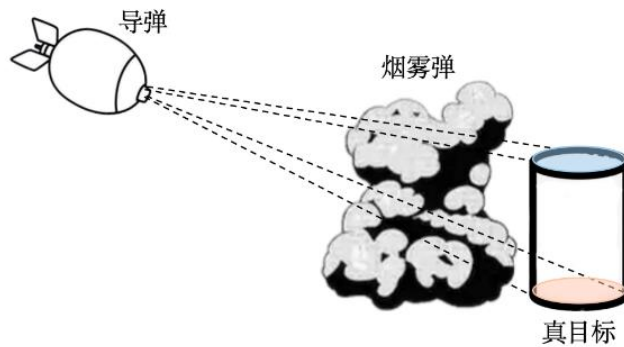


图 2. 烟雾云团遮盖真目标圆柱体上下表面示意图

如果线段与云团相交，则光线路径被阻挡。所有采样点均被遮蔽时，认为导弹被完

全遮蔽。随后我们通过下面的公式来判定， $s = 0$ 对应导弹位置， $s = 1$ 对应目标表面点：

$$(P_{Center}^{(1,1)}(t) - (M_1(t) + s(P_i - M_1(t))))^2 \leq R^2, \quad 0 \leq s \leq 1$$

联立导弹与烟幕云团的三维相对运动模型与完全遮蔽几何判断模型如下：

$$\begin{cases} M_1(t) = M_1(0) + v_{M_1} \cdot t \cdot u_{M_1} \\ P_{explode}^{(1,1)} = P_{drop}^{(1,1)} + v_{U_1} \cdot \tau_{fuse}^{(1,1)} \cdot u_{U_1} + \frac{1}{2} \cdot g \cdot (\tau_{fuse}^{(1,1)})^2 \cdot k \\ t_{explode}^{(1,1)} = t_{rel}^{(1,1)} + \tau_{fuse}^{(1,1)} \\ P_{Center}^{(1,1)}(t) = P_{explode}^{(1,1)} - v_s \cdot (t - t_{explode}^{(1,1)}), \quad t \in [t_{explode}^{(1,1)}, t_{explode}^{(1,1)} + 20] \\ \left(P_{Center}^{(1,1)}(t) - (M_1(t) + s(P_i - M_1(t))) \right)^2 \leq R^2, \quad 0 \leq s \leq 1 \end{cases}$$

对时间区间 $[t_{explode}^{(1,1)}, t_{explode}^{(1,1)} + 20]$ 以小步长采样，生成布尔序列：

$$cover(t) = \begin{cases} True, & \text{导弹完全遮蔽} \\ False, & \text{导弹未遮蔽} \end{cases}$$

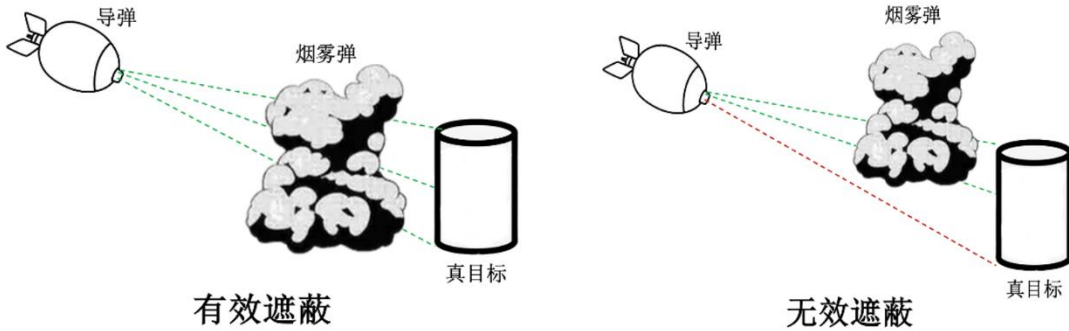


图 3. 烟幕云团有效和无效遮盖示意图

统计导弹被完全遮蔽的连续区间。我们通过二分法精细求解每个遮蔽区间的起止时间，并累加求得总遮蔽时长：

$$T_{cover}^1 = \sum_i (t_{end}^{(i)} - t_{start}^{(i)})$$

最终通过求解导弹与烟幕云团的三维相对运动模型与几何遮蔽判断模型，我们得到无人机 FY1 投放烟幕干扰弹对导弹 M1 的有效遮蔽时长约为 **1.4050s**，同时可得到具体遮蔽时间区间 **[8.043187, 9.448222]s**。

5.2 问题二模型的建立与求解

利用无人机 FY1 投放 1 枚烟幕干扰弹对来袭导弹 M1 进行干扰，我们的目标是最大化烟幕干扰弹形成的遮蔽时间，使得导弹 M1 在飞行过程中尽量长时间处于烟幕的有效遮蔽范围内。

此时的 FY1 的飞行方向 θ_{U_1} 、飞行速度 v_{U_1} 、烟幕干扰弹投放点 $P_{drop}^{(1,1)}$ 、烟幕干扰弹

起爆点 $P_{explode}^{(1,1)}$ 都是我们的决策变量，根据问题一的3D动力学模型计算遮蔽时间。

本题我们利用退火改进粒子群优化（SA-PSO）方法，将 SA 的温度-退火机制与 PSO 的群体搜索结合，优化无人机的飞行策略，实现遮蔽时间的最大化。

5.2.1 单导弹·单无人机·单烟幕弹[1 1 1]最大遮蔽时间优化模型的建立

分析问题二的任务可知，我们需要对单导弹，单无人机，单烟幕弹的情形进行优化决策，接下来我们将这种情况的优化模型简称为[1 1 1]最大遮蔽时间优化模型。

目标函数：

在本问题中，目标函数是遮蔽时长的负值，因为 PSO 是一个最小化问题，因此我们将遮蔽时长反向作为适应度函数。为了最大化遮蔽时长 T^1_{cover} ，我们定义目标函数为：

$$f(x) = \min(-T^1_{cover})$$

决策变量：

$$X_i = [\theta_i, v_{U_i}, P_{drop}^{(i,j)}(t_{rel}^{(i,j)}), P_{explode}^{(i,j)}(\tau_{fuse}^{(i,j)})]$$

约束条件：

■ 航向角 θ_1 范围：

表示 FY1 的飞行方向，单位为弧度（rad），取值范围为 $[0, 2\pi)$ ，即无人机可在水平面内任意方向飞行。该变量直接影响投放点的空间位置。

通过几何观测易得出无人机以及导弹的方位可知要使得遮蔽时间最大，烟幕弹下落时圆心需要经过导弹运行轨迹以及真目标的线段上。因此可以给航向角设定一定的预测最佳范围，减少搜索的范围，增加决策的正确率。开始时我们将航向角的范围设定在一定较大范围内，随着结果的收敛，我们逐渐缩小航向角的范围，得到的优化结果较佳。（根据实际测试情况调整范围）

■ 飞行速度 v_{U1} 范围：

根据题目要求，无人机的飞行速度必须在规定范围内， $70m/s \leq v_{U1} \leq 140m/s$ 。该变量决定了无人机到达投放点的速度，从而影响烟幕弹的初始运动。

■ 投放点时间 $t_{rel}^{(1,1)}$ 范围：

烟幕干扰弹的投放时间，单位为秒(s)，从任务受领时刻起计算，取值范围为 $[0, t_{max}]$ ，放延迟 $t_{rel}^{(1,1)}$ 应该非负，其中 $t_{max}(66.67s)$ 为导弹到达假目标的时间。

■ 引信延迟 $\tau_{fuse}^{(1,1)}$ 范围：

引信延迟需要满足非负， $0 \leq \tau_{fuse}^{(1,1)}$ ，并且通过查阅相关资料，我们发现烟幕弹的引线延迟时间可根据情景不同有较大的范围，因此我们的最大上限时间设计的范围较宽泛。

■ 烟幕弹运动轨迹:

烟幕干扰弹起爆前后下落轨迹和速度不能超过一定限制,例如起爆前烟幕弹不可落地否则无法进行有效遮蔽,同时起爆后下落速度和轨迹由于空气阻力受到限制,并且需要最大化烟雾弹起爆后在一定时间内对真目标进行有效遮蔽的时间。

综上[1 1 1]最大遮蔽时间优化模型需要满足以下约束条件:

$$s.t. \begin{cases} \theta_1 \in [0, 2\pi) \\ 70m/s \leq v_{U1} \leq 140m/s \\ 0s \leq t_{rel}^{(1,1)} \leq 66/67s \\ 0s \leq \tau_{fuse}^{(1,1)} \\ 0m \leq P_{explode}^{(1,1)}, z \end{cases}$$

因此,我们可以确定[1 1 1]最大遮蔽时间优化模型为:

$$f(x) = \min(-T^1_{cover})$$

目标函数:

$$\begin{cases} M_1(t) = M_1(0) + v_{M_1} \cdot t \cdot u_{M_1} \\ U_1(t) = U_1(0) + v_{U_1} \cdot t \cdot u_{U_1} \\ P_{drop}^{(1,1)} = U_1(t_{rel}^{(1,1)}) = U_1(0) + v_{U_1} \cdot t_{rel}^{(1,1)} \cdot u_{U_1} \\ P_{explode}^{(1,1)} = P_{drop}^{(1,1)} + v_{U_1} \cdot \tau_{fuse}^{(1,1)} \cdot u_{U_1} + \frac{1}{2} \cdot g \cdot (\tau_{fuse}^{(1,1)})^2 \cdot k \\ t_{explode}^{(1,1)} = t_{rel}^{(1,1)} + \tau_{fuse}^{(1,1)} \\ P_{Center}^{(1,1)}(t) = P_{explode}^{(1,1)} - v_s \cdot (t - t_{explode}^{(1,1)}), \quad t \in [t_{explode}^{(1,1)}, t_{explode}^{(1,1)} + 20] \\ \left(P_{Center}^{(1,1)}(t) - (M_1(t) + s(P_i - M_1(t))) \right)^2 \leq R^2, \quad 0 \leq s \leq 1 \\ cover(t) = \begin{cases} True, \text{导弹完全遮蔽} \\ False, \text{导弹未遮蔽} \end{cases} \\ T^1_{cover} = \sum_i (t_{end}^{(i)} - t_{start}^{(i)}) \\ f(x) = \min(-T^1_{cover}) \end{cases}$$

决策变量:

$$X_1 = [\theta_1, v_u, P_{drop}^{(1,1)}(t_{rel}^{(1,1)}), P_{explode}^{(1,1)}(\tau_{fuse}^{(1,1)})]$$

约束条件:

$$\begin{cases} \theta_1 \in [0, 2\pi) \\ 70m/s \leq v_{U1} \leq 140m/s \\ 0s \leq t_{rel}^{(1,1)} \leq 66/67s \\ 0s \leq \tau_{fuse}^{(1,1)} \\ 0m \leq P_{explode}^{(1,1)}, z \end{cases}$$

5.2.2 单导弹·单无人机·单烟幕弹[1 1 1]最大遮蔽时间优化模型的求解

对于问题二的优化，我们面临多个相互依赖的决策变量，无人机的飞行速度、航向角、烟幕投放点和起爆点，这些变量间存在强烈的非线性关系，导致我们的目标函数：

$$f(x) = \min(-T_{cover}^1)$$

该目标函数高度复杂且可能存在多个局部最优。传统优化方法如梯度下降（GD），依赖梯度信息，无法直接处理非连续约束，且对初始点极度敏感，容易陷入局部最优。模拟退火（SA）虽具备跳出局部的理论优势，但在本问题中，由于目标函数的能量障碍较高，SA 在多峰高维空间中越过高阻隔的概率极低，收敛效率不理想。

相比之下，粒子群优化（PSO）通过群体协作与个体经验相结合，能在无梯度信息的情况下进行全局搜索，具有较好的鲁棒性。然而，标准 PSO 易发生早熟收敛，在复杂高维搜索空间中陷入局部最优。

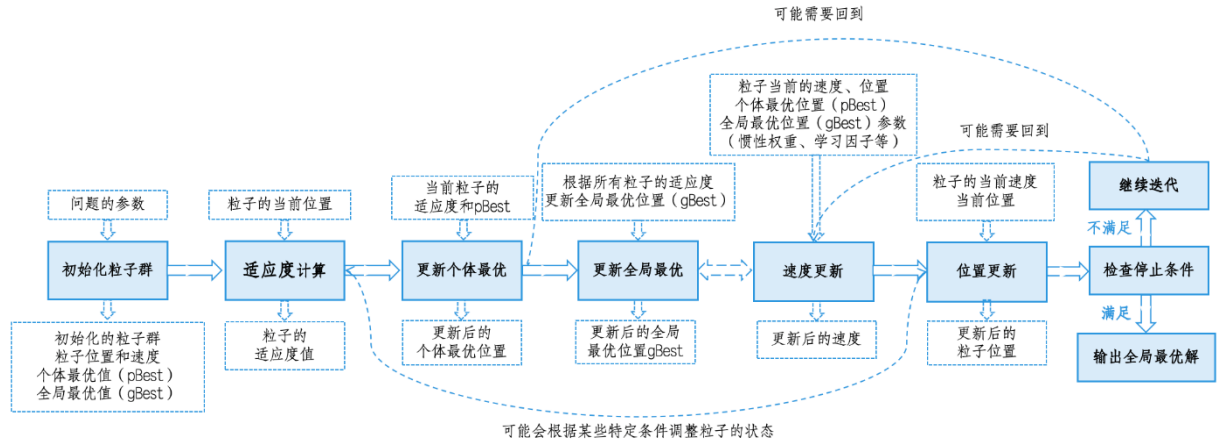
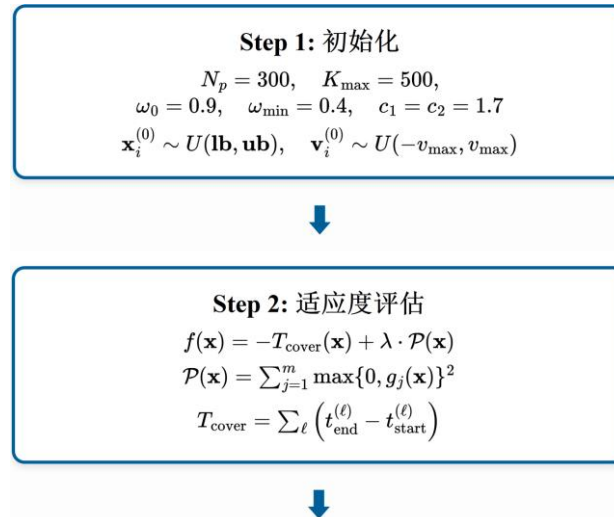
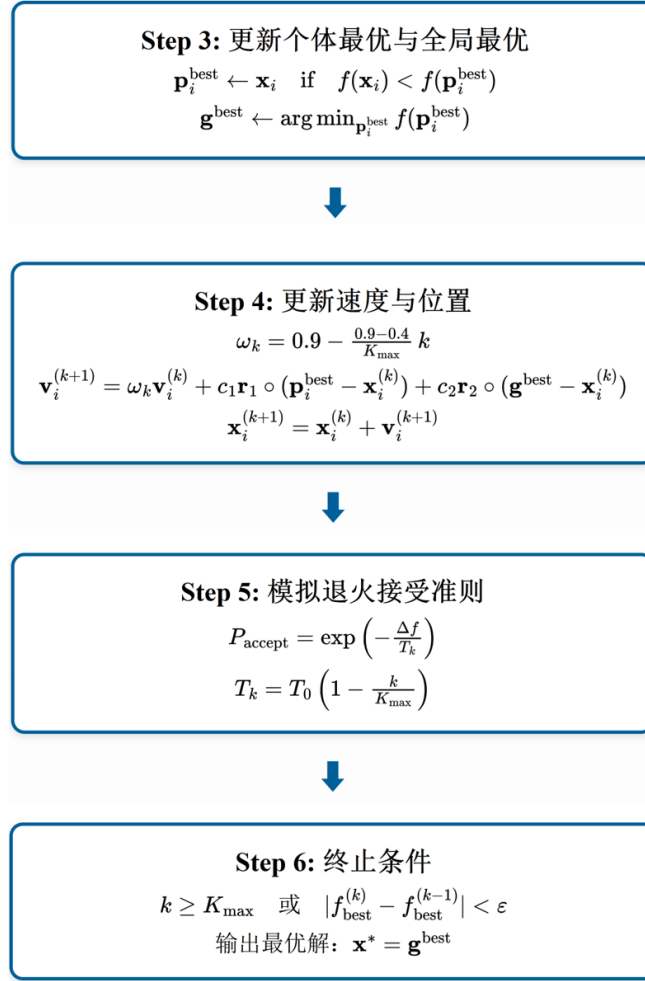


图 4. 粒子群 (PSO) 算法流程图

因此，我们选用退火改进粒子群优化（SA-PSO）方法，将 SA 的温度-退火机制与 PSO 的群体搜索结合，以平衡全局探索与局部收敛能力。

以下是我们退火改进粒子群优化（SA-PSO）组合优化算法求解的具体思路：





经过多次迭代计算后，我们最终得到理想飞行策略如下：

决策参数	优化结果
FY1 航向角 θ_1	0.1255rad(7.19°)
FY1 飞行速度 v_{U1}	131.89m/s
烟幕弹投放时间 $t_{rel}^{(i,j)}$	0.000s
引信延迟 $\tau_{fuse}^{(1,1)}$	0.739s
烟幕弹投放位置坐标 $P_{drop}^{(1,1)}$	(17800.00, 0.00, 1800.00) m
起爆点 $P_{explode}^{(1,1)}$	(17896.68, 12.20, 1797.32)m
起爆时间 $t_{explode}^{(1,1)}$	0.739 s
最大完全遮蔽时长 T_{cover}^1	4.590 s

表 2. 理想飞行策略决策参数记录表

5.2.2 单导弹·单无人机·单烟幕弹[1 1 1]最大遮蔽时间优化模型的验证

我们设定决策参数为 PSO 优化的最佳结果：FY1 航向角 $\theta_1 = 0.1255rad(7.19^\circ)$ ，FY1 飞行速度 $v_{U1} = 131.89m/s$ ，烟幕弹投放时间 $t_{rel}^{(i,j)} = 0.000s$ ，引信延迟 $\tau_{fuse}^{(1,1)} = 0.739s$ ，通过带入问题一的三维运动学方程进行独立验证实验，并分析 PSO 算法的收敛过程，来证明我们所建立的优化模型的合理性和准确性。

下图展示了 PSO 迭代过程中最佳遮蔽时长的收敛曲线，图中最优值收敛，接近 4.590 s，以及轨迹模拟下的遮蔽时长计算结果。

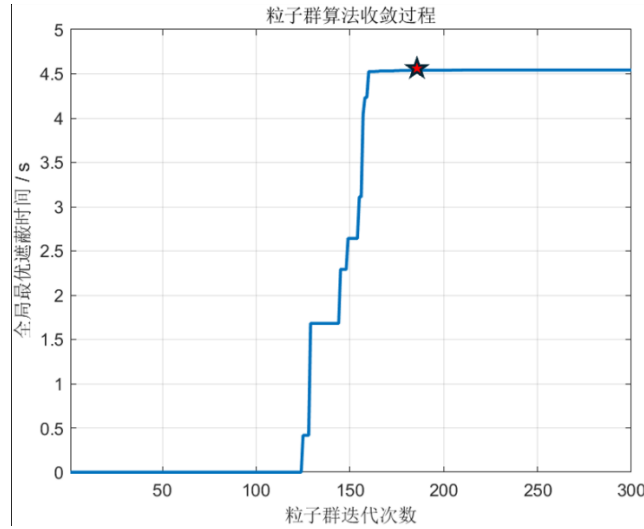


图 5. PSO 迭代收敛曲线

如图 5 所示，SA - PSO 优化过程中，完全遮蔽时长由初始的 0.000 快速提升并在 $k \approx 150$ 时收敛至 4.590 s，满足起爆高度与时间窗等约束。独立验证结果为 4.5847 s，误差 $< 0.12\%$ ，遮蔽区间为 $[0.919426, 5.504174] s$ 。相比问题一中无优化策略的 1.405 s，提升约 226.8%，验证了该方法的有效性与快速收敛性。

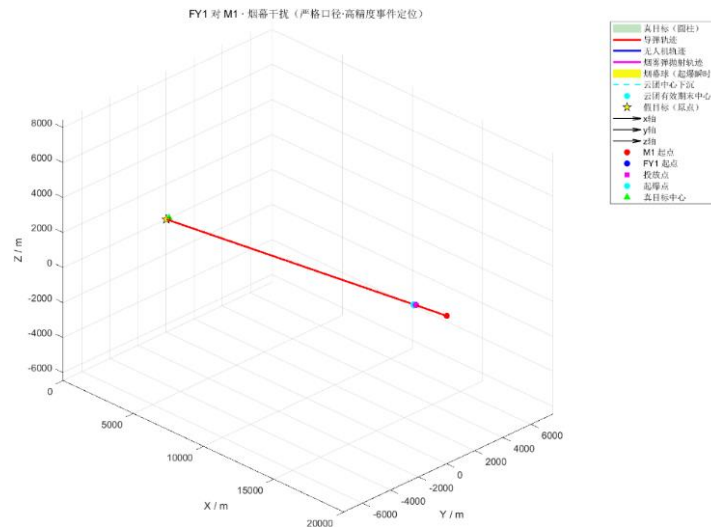


图 6. 导弹、烟幕弹轨迹模拟示意图

如图 6、图 7 所示，云团半径 10 m 的球形区域可完全遮蔽真目标采样点，满足“所有视线与云团相交”的判定条件。 PSO 在约 160 次迭代内由 0.000 s 快速提升至约 4.000 s ，并于 $k \approx 150$ 后趋于平稳，最终收敛至接近 4.590 s 。该结果体现了 PSO 在非线性、多峰优化中的全局搜索与稳定性。误差主要来自时间步长 $dt = 0.01\text{ s}$ 的离散化及算法随机性，但多次运行结果一致，验证了模型的鲁棒性与物理可靠性。

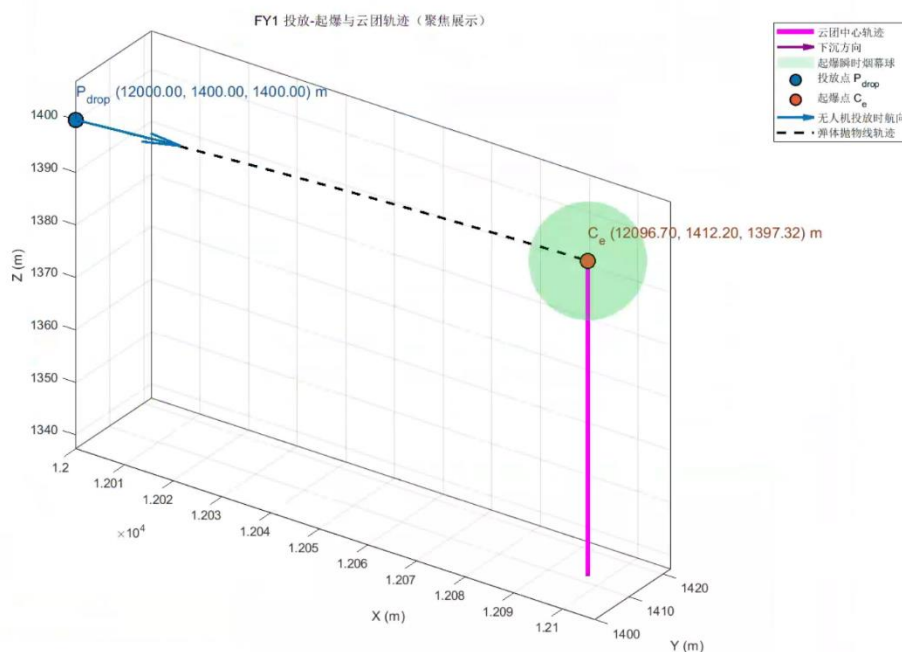


图 7. 局部导弹、烟幕弹轨迹模拟示意图

5.3 问题三模型的建立与求解

问题三的任务是利用无人机 FY1 投放三枚烟幕干扰弹，对导弹 M1 实施干扰，制定投放时刻、起爆时刻及飞行参数，即航向和速度，以最大化烟幕对 M1 的有效遮蔽总时长。核心挑战在于多弹投放的时空协调，避免重叠浪费并优化累积遮蔽区间。

我们沿用了上述三维运动学模型，构建多阶段时空覆盖模型，模拟导弹与各烟幕的相对运动。优化目标函数为总遮蔽时长，考虑约束。采用差分进化算法 (DE) 进行全局搜索，初始化种群并通过变异、交叉和选择操作生成解；随后，使用粒子群优化 (PSO) 进行局部抛光，精细调整参数。最终结果包括投放序列，验证通过迭代收敛曲线，确保策略的鲁棒性和效率。

5.3.1 单导弹·单无人机·多烟幕弹[1 1 3]最大遮蔽时间优化模型的建立

分析问题三的任务可知，我们需要对单导弹，单无人机，多烟幕弹的情形进行优化决策，同问题二我们将这种情况的优化模型简称为[1 1 3]最大遮蔽时间优化模型。导弹、烟幕弹及无人机的运动模型沿用上述题目的建模；几何采样点的选取方法亦与此前随机采样的方法一致。

因此，我们可以确定[1 1 3]最大遮蔽时间优化模型为：

$$\max(J) = \max(T_{cover}^1) = \max\left(\sum_{k=1}^3 \text{有效遮蔽时段}_k \cdot 3 \text{枚烟幕弹}\right) - T_{overlap}$$

目标函数：

$$\left\{ \begin{array}{l} M_1(t) = M_1(0) + v_{M_1} \cdot t \cdot u_{M_1} \\ U_1(t) = U_1(0) + v_{U_1} \cdot t \cdot u_{U_1} \\ P_{drop}^{(1,j)} = U_1(t_{rel}^{(1,j)}) = U_1(0) + v_{U_1} \cdot t_{rel}^{(1,j)} \cdot u_{U_1} \\ P_{explode}^{(1,j)} = P_{drop}^{(1,j)} + v_{U_1} \cdot \tau_{fuse}^{(1,j)} \cdot u_{U_1} + \frac{1}{2} \cdot g \cdot (\tau_{fuse}^{(1,j)})^2 \cdot k \\ t_{explode}^{(1,j)} = t_{rel}^{(1,j)} + \tau_{fuse}^{(1,j)} \\ P_{Center}^{(1,j)}(t) = P_{explode}^{(1,j)} - v_s \cdot (t - t_{explode}^{(1,j)}), \quad t \in [t_{explode}^{(1,j)}, t_{explode}^{(1,j)} + 20] \\ \left(P_{Center}^{(1,j)}(t) - (M_1(t) + s(P_i - M_1(t))) \right)^2 \leq R^2, \quad 0 \leq s \leq 1 \\ cover(t) = \begin{cases} True, & \text{导弹完全遮蔽} \\ False, & \text{导弹未遮蔽} \end{cases} \\ T_{cover}^1 = \sum_i (t_{end}^{(i)} - t_{start}^{(i)}) \\ \max(J) = \max(T_{cover}^1) = \max\left(\sum_{k=1}^3 \text{有效遮蔽时段}_k \cdot 3 \text{枚烟幕弹}\right) \end{array} \right.$$

其中 $k = 1, 2, 3$ 对应于三枚烟幕干扰弹。每一枚烟幕干扰弹有一个投放点、起爆点以及目标之间的遮蔽时段。通过优化烟幕干扰弹的投放时间与起爆时间，我们想让这些遮蔽时段的和尽可能地长。

决策变量：

$$X_1 = [\theta_1, v_{U_1}, P_{drop}^{(1,j)}(t_{rel}^{(1,j)}), P_{explode}^{(1,j)}(\tau_{fuse}^{(1,j)})]$$

其中 $j = 1, 2, 3$ 对应于 FY1 投掷的 3 枚烟幕干扰弹。

约束条件：

根据题目二的分析我们根据已知的约束条件上增加一条约束。

■ 不同烟幕弹的投放间隔

根据题目已知每两枚烟幕干扰弹的投放时间间隔必须至少为 1 秒

$$\begin{aligned} t_{rel,k+1} - t_{rel,k} &\geq 1, & k &= 1, 2 \\ 0 &\leq t_{rel,k} \leq t_{max}(66.67s), & k &= 1, 2, 3 \end{aligned}$$

综上[1 1 3]最大遮蔽时间优化模型的约束条件如下：

$$s. t. \begin{cases} \theta_1 \in [0, 2\pi) \\ 70m/s \leq v_{U_1} \leq 140m/s \\ 0s \leq t_{rel}^{(1,j)} \leq 66.67s, j = 1, 2, 3 \\ 0s \leq \tau_{fuse}^{(1,j)}, j = 1, 2, 3 \\ 0m \leq P_{explode, z}^{(1,j)}, j = 1, 2, 3 \\ t_{rel}^{(1,k+1)} - t_{rel}^{(1,k)} \geq 1s, k = 1, 2 \end{cases}$$

5.3.2 单导弹·单无人机·多烟幕弹[1 1 3]最大遮蔽时间优化模型的求解

本问题在多约束条件下求解非线性、非凸、多峰的高维优化目标。差分进化（DE）全局搜索能力强，善于处理非线性耦合与多峰问题，通过罚函数可有效剔除无效解，避免陷入局部最优；粒子群优化（PSO）局部收敛速度快，但高维全局搜索效率较低。

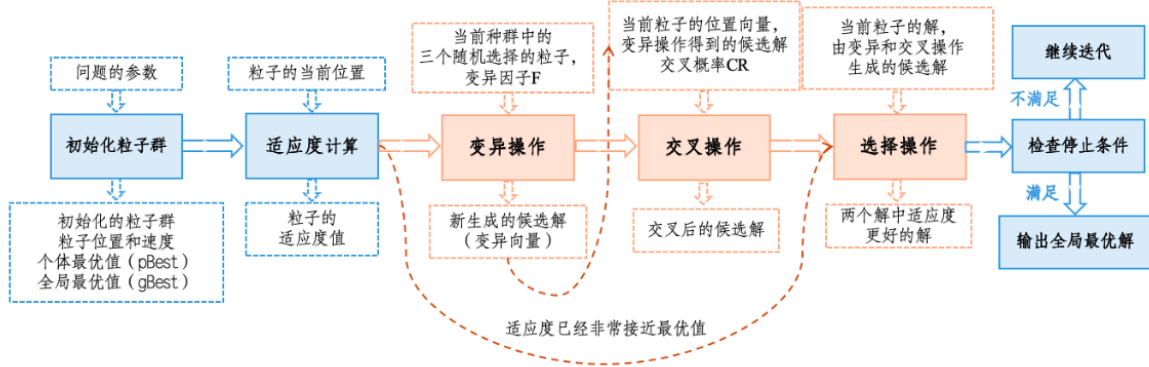


图 8. 差分进化算法逻辑示意图

因此采用 **DE 全局搜索 + PSO 局部精修** 的组合优化策略，以兼顾全局探索与局部精确优化，提升求解稳定性与精度。

以下为我们“DE 全局搜索+ PSO 局部精修”组合优化算法具体求解过程：

Step 1. 初始化种群。随机生成 $x_i^{(0)}, i = 1, \dots, N$ ，满足约束条件。

Step 2. 适应度计算。目标函数为有效遮蔽时间：

$$\max f(x) = \sum_{k=1}^m T_k(x)$$

Step 3. 变异操作。从种群中选取互易个体 x_a, x_b, x_c ：

$$v_i = x_a + F \cdot (x_b - x_c), F \in (0, 1)$$

Step 4. 交叉与选择。生成候选解 y_i ，采用随机选择的方法，并比较适应度：

$$x_i = \begin{cases} y_i & , f(y_i) \geq f(x_i) \\ x_i^{(t)} & , \text{otherwise} \end{cases}$$

Step 5. 收敛检测。停止条件为以下判断满足其一即可：

$$t \geq G_{max}$$

$$\left| f_{max}^{(t)} - f_{max}^{(t-1)} \right| < \varepsilon$$

Step 6. 粒子群局部精修。以 DE 最优解为初值按照粒子群优化更新：

$$v_i^{k+1} = wv_i^k + c_1r_1(p_i^{best} - x_i^k) + c_2r_2(p_g^{best} - x_i^k)$$

$$x_i^{k+1} = x_i^k + v_i^{k+1}$$

最终得到精细化全局最优解。决策结果如下，并且将结果保存在 result1.xlsx 文件中。

无人机的运动方向为 7.152456° ，速度方向为 $112.598919m/s$ 。

烟幕弹 编号	烟幕干扰弹投放点的坐标 (m)	烟幕干扰弹起爆点的x坐标 (m)	有效干扰时长 (s)
1	(17800, 0, 1800)	(17800, 0, 1800)	2.729
2	(17911.72, 14.020, 1800)	(17911.72, 14.020, 1800)	4.747
3	(19084.07, 161.13, 180)	(19084.07, 161.13, 1800)	无

表 3. 无人机飞行投掷位置参数及有效遮蔽时长记录表

我们为了更直观地观察无人机、导弹和烟雾弹的运动轨迹和烟雾弹的状态，我们在三维空间中画图表示，如下图 9 所示。

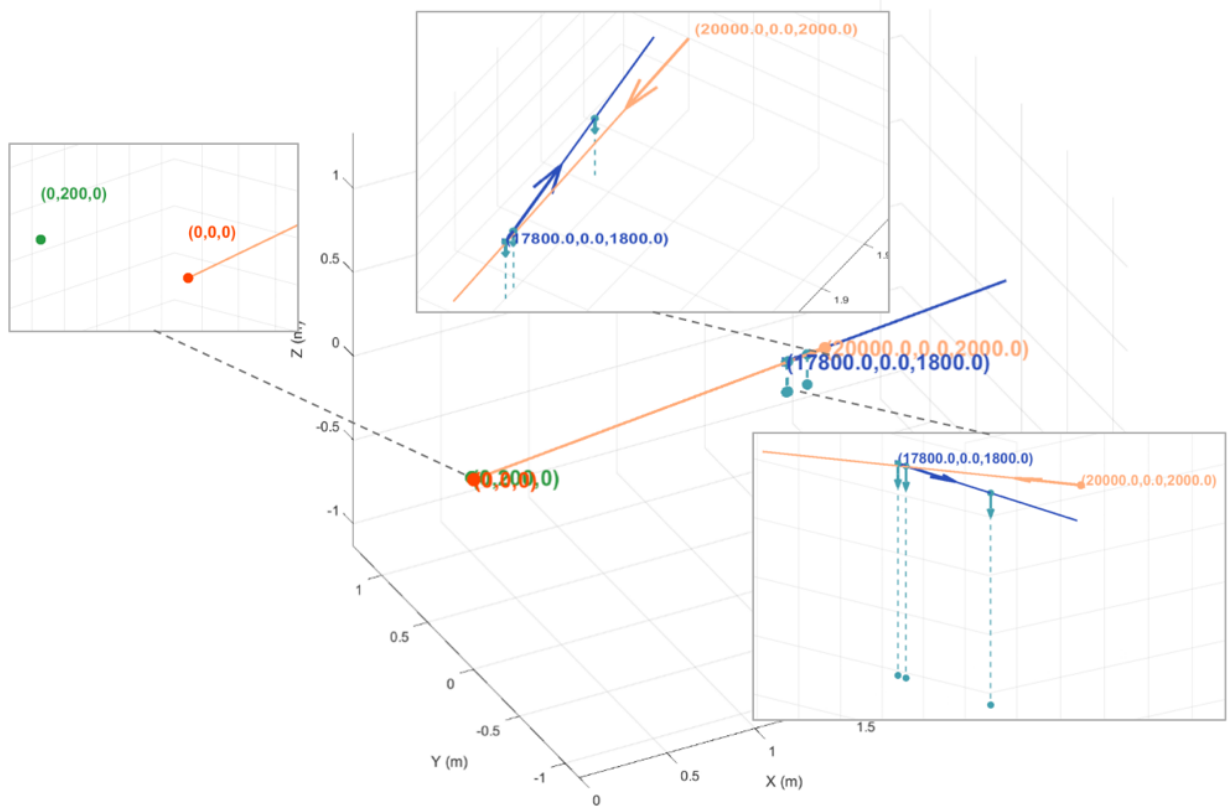


图 9.导弹、烟幕弹运动轨迹示意图

深蓝色轨迹：无人机的飞行路径，箭头表示运动方向；

橙色轨迹：导弹的飞行轨迹，箭头表示运动方向；

蓝绿色轨迹：烟幕干扰弹在起爆后以恒定速度下沉的运动路径，箭头表示运动方向。

绿色圆点与红色圆点分别标示**真目标与假目标**下底面圆心位置，便于直观判断烟幕干扰弹在导弹视线范围内对真目标的有效遮蔽情况。

在本次优化求解结果中，无人机在到达预定投放位置后立即投放并起爆烟幕干扰弹。由于起爆与投放位置一致，图中以同一位置点同时表示干扰弹的投掷点与起爆点。起爆后，烟幕云团沿竖直方向以恒定速度均匀下沉，并在有效遮蔽时间内保持球形扩散形态。通过图 9，可以清晰地观察无人机与导弹在空间中的相对运动状态，以及烟幕云团在有效遮蔽阶段的空间位置与覆盖效果，通过观察，我们对求解结果进行**时间重叠分析**。

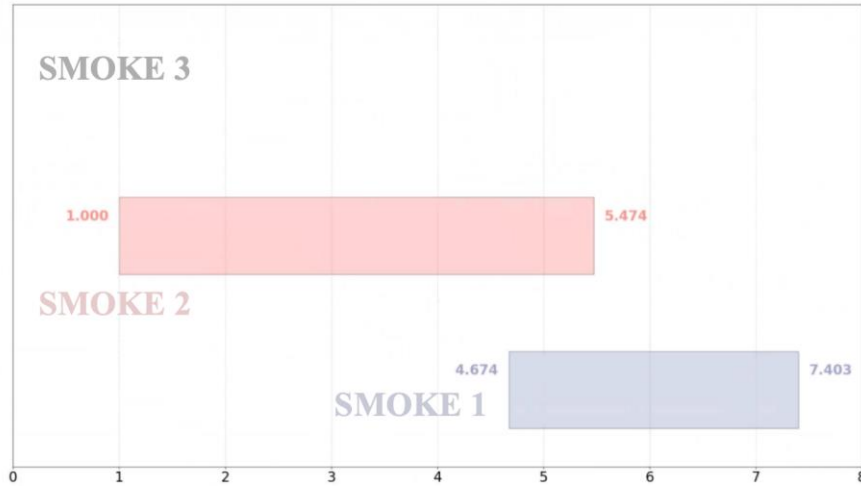


图 10. 时间重叠分析示意图

图 10 展示了单架无人机分别投放三枚烟雾弹并引爆烟雾弹所产生的遮蔽效应在时间轴上的分布情况。横轴为时间，纵向不同行代表投放不同的烟雾弹。彩色矩形区域表示烟雾弹在引爆后产生的有效遮蔽时间区间。观察可知烟幕弹 1 和 2 在时空上小范围重叠，最大有效遮蔽时长应减去重叠部分，优化得到的最大有效遮蔽时长为**6.403159s**。

$$I_1 \cap I_2 \neq \emptyset$$

$$I_1 = [1.000, 5.474]s$$

$$I_2 = [4.674, 7.403]s$$

$$T_{cover}^1 = I_1 \cup I_2 = 6.403159s$$

5.3.2 单导弹·单无人机·多烟幕弹[1 1 3]最大遮蔽时间优化模型的验证

我们记录***J***最长屏蔽时间最着迭代的最优解数值曲线，从图中可以看出，随着算法的迭代进行，最遮蔽时间逐渐趋于平稳，并且变化幅度越来越小。这表明，随着代数的增加，算法已经成功地找到了最优解，并且在之后的迭代中，解的质量几乎没有显著波动，呈现出平稳的收敛趋势。具体来说，图中的适应度值随着代数的增加不断上升，并在一

定时刻趋于平稳，说明算法已经找到并稳定在局部最优解附近。

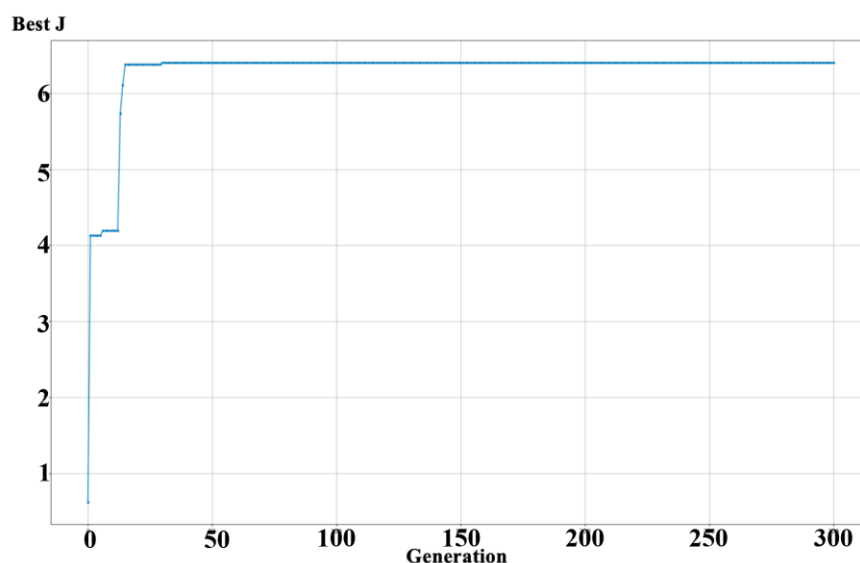


图 11. 最优解数值曲线

如图平稳收敛，进一步证明了算法在搜索解空间时能够有效地避免过度震荡和局部最优解的困扰。通过差分进化 **DE** 和粒子群优化 **PSO** 的结合，本算法能够在全局搜索和局部优化之间找到一个平衡点，确保算法在复杂的多维解空间中稳定地逼近最优解。这种收敛性验证了算法的有效性，并确保最终求得的最优遮蔽时间是稳定且可靠的。

5.4 问题四模型的建立与求解

问题四的任务是利用三架无人机（FY1~FY3）各投放一枚烟幕干扰弹，对导弹 M1 实施协同干扰，制定多无人机投放策略，包括各无人机的飞行方向、速度、投放点和起爆点，以最大化烟幕对 M1 的总遮蔽时间。核心挑战在于多平台的空间时序协调，避免遮蔽空档或重叠浪费，并考虑各无人机初始位置差异。

构建单导弹·多无人机·单烟幕弹[131]最大遮蔽时间优化模型，各无人机独立求解飞行参数，但需全局协调以实现协同覆盖。分析模拟导弹、无人机以及烟幕弹的相对运动轨迹，验证时空不交叠，我们确保它们之间互不干扰。最终求解分解为子问题，整合策略以最大化总遮蔽时间，并输出重要参数结果，即投放、起爆坐标和有效遮蔽时长，验证通过轨迹图和收敛分析，确保鲁棒性。

5.4.1 单导弹·多无人机·单烟幕弹[131]最大遮蔽时间优化模型的建立

基于问题四的任务，即利用三架无人机(FY1、FY2、FY3)各投放一枚烟幕干扰弹，对导弹 M1 实施协同干扰，我们建立单导弹多无人机单烟幕弹[131]最大遮蔽时间优化模型。该模型扩展了问题二的[111]模型(单无人机单烟幕弹优化)和问题三的[113]模型(单无人机多烟幕弹优化)，重点强调多平台协同的时空协调。

由于飞机的飞行高度固定不变，已知：

$$U_{1,z}(t) = 2000$$

$$U_{2,z}(t) = 2100$$

$$U_{3,z}(t) = 1900$$

由此可得：

$$U_{1,z}(t) \neq U_{2,z}(t) \neq U_{3,z}(t)$$

因此，无人机 FY1、FY2、FY3 三机分别在平行的不同等高平面上飞行，满足垂直间隔要求，**可忽略相互碰撞风险**；在运动学约束层面可视为互不约束、相互独立。基于此，我们先对每架飞机独立求解其水平机动，即朝向与速度，以及投放、引爆时刻的最优组合，以最大化其对 M1 导弹提供的单机遮蔽时长，从而得到对应的候选遮蔽时间区间；也就是说，我们可以分别求第*i*架飞机对于导弹 M1 的最长遮蔽时间 $T_{cover}^{(i,1)}$ 。在上述条件成立时，问题四的求解可等价分解为三个彼此独立的“问题一”子问题，即

$$\max (T_{cover}^1) = T_{cover}^{(1,1)} + T_{cover}^{(2,1)} + T_{cover}^{(3,1)}$$

● 验证时空不交叠：贪心结论成立

需要指出的是，“局部最优解叠加即可得到全局最优解”的贪心结论仅在三段遮蔽区间在时间轴上基本不重叠、或重叠对总遮蔽时长无额外贡献时才成立；一旦存在显著时间重叠，上述做法会高估总遮蔽时间，局部最优解不一定构成全局最优解。因此，我们分别求解三架无人机对应的遮蔽时间区间；该过程与问题一的模型一致，代入各自参数条件后得到三个区间 $I_i = [t_{start}^{(i)}, t_{end}^{(i)}]$ 。若满足 $\forall i \neq j, I_i \cap I_j = \emptyset$ ，则在此条件下，**贪心算法成立**，总遮蔽时长等于三段区间长度之和，即局部最优解之和为全局最优解。

此外我们还需要验证各个烟幕投放弹的运动与各个无人机的运动互不干扰，即不出现**时空交叠**的情况。

5.4.2 单导弹·多无人机·单烟幕弹[1 3 1]最大遮蔽时间优化模型的求解

根据问题二建立的[1 1 1]最大遮蔽时间优化模型，分别对三架飞机单独进行时序优化决策。FY1 的最大遮蔽时长在第二问已经求解，若要求解 FY2、FY3 的遮蔽时长，我们仍利用子问题二的优化模型。通过求解目标函数 $f(x) = \min(-T_{cover}^1)$

我们得到以下决策参数以及结论：

无人机编号	飞行角度	飞行速度 (m/s)	投放点(m)	起爆点(m)
<i>FY1</i>	7.19°	131.89	(17800,0,1800)	(179896.68, 12.2, 1797.33)
<i>FY2</i>	296.18°	113.18	(12417.87, 550.18, 1400)	(12671.06, 3529, 1273.93)
<i>FY3</i>	75.98°	104.49	(6572.52, -115.23, 700)	(6767.08, 71.17, 683.42)

表 4. 最佳无人机飞行投掷决策参数记录表 1

$$T_{cover}^{(1,1)} = 4.5844s, I_1 = [0.906324, 5.490680]$$

$$T_{cover}^{(2,1)} = 3.9350s, I_2 = [13.513528, 17.448511]$$

$$T_{cover}^{(3,1)} = 2.9723s, I_3 = [30.386445, 33.358684]$$

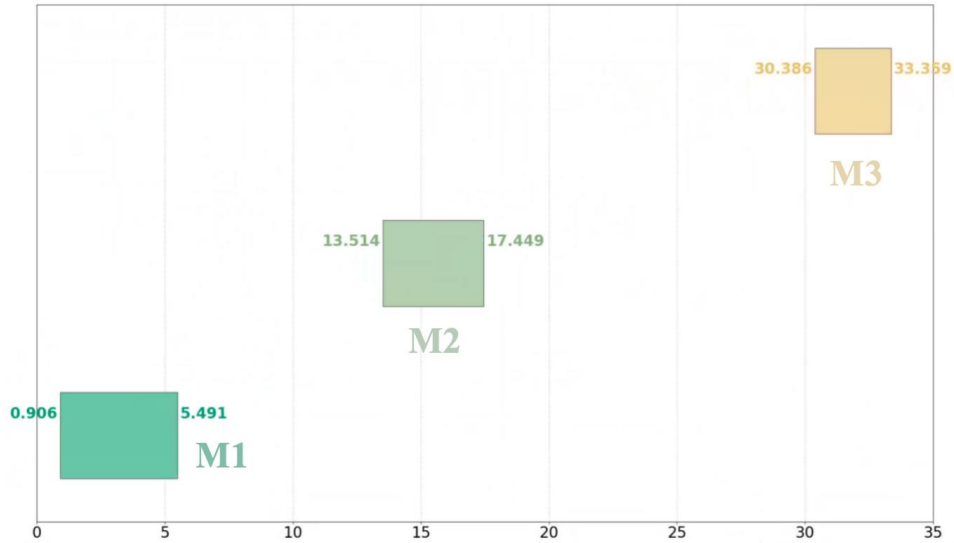


图 12. 时间重叠分析示意图

图 12 展示了三架无人机分别投放并引爆烟雾弹所产生的遮蔽效应在时间轴上的分布情况。横轴为时间，纵向不同列行代表一架无人机投放的烟雾弹。彩色矩形区域表示烟雾弹在引爆后产生的有效遮蔽时间区间，不同颜色对应不同无人机的投放任务。观察可得三架飞机单独优化得到得时间互补交叠，满足 $\forall i \neq j, I_i \cap I_j = \emptyset$ ，因此可用贪心结论得到：

$$\max(T_{cover}^1) = T_{cover}^{(1,1)} + T_{cover}^{(2,1)} + T_{cover}^{(3,1)} = 11.4917s$$

浅蓝色直线：表示无人机的飞行轨迹，标注“FY1”“FY2”“FY3”分别对应三架无人机，箭头用于指示飞行方向；

浅红色直线：表示来袭导弹的飞行轨迹，从右下角“导弹起点”沿直线飞向真目标；

深红色线直线：表示烟幕干扰弹在起爆后沿竖直方向匀速下沉的运动路径；

绿色圆点与灰色圆点：分别标示真目标与假目标下底面圆心位置，用于直观判断烟幕云团在导弹视线范围内的遮蔽效果。

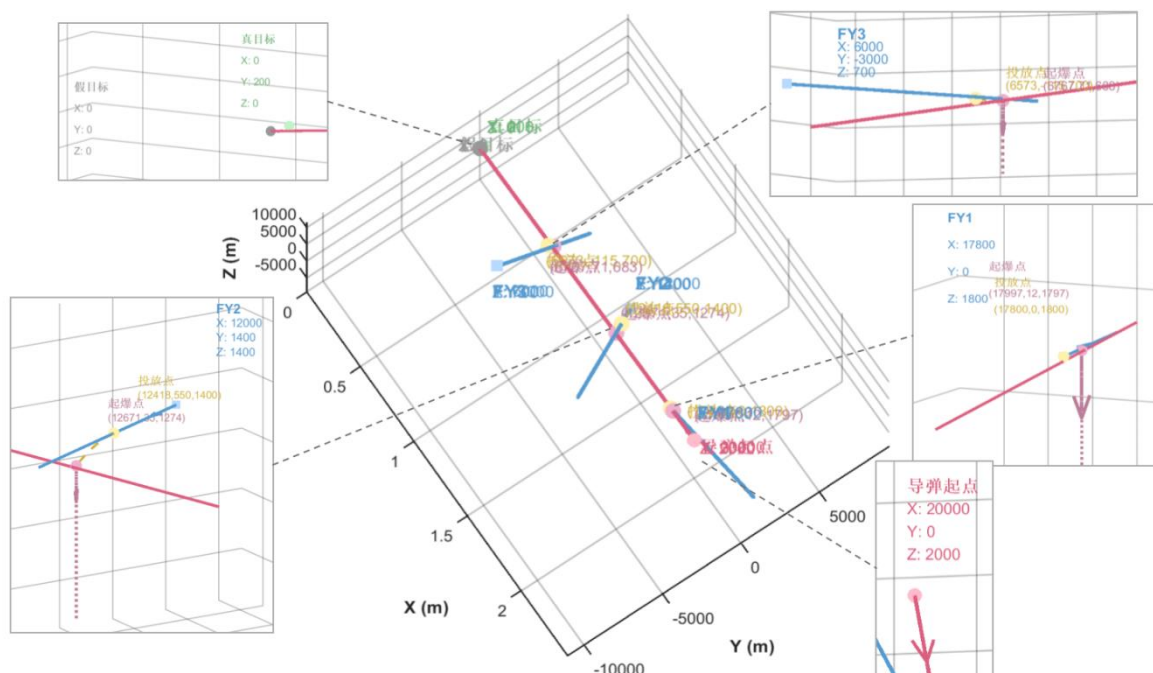


图 13 导弹、多无人机投掷烟幕弹运动轨迹示意图

图 13 展示了无人机、导弹及烟幕干扰弹在三维空间中的相对运动关系。为便于分析关键事件，图中设置了五个放大视窗。

左上角：真目标与假目标的平面位置对比，用于评估烟幕云团的遮蔽覆盖效果；

左下角：无人机 FY2 的投放位置与起爆位置（重合），可见烟幕沿垂直方向的下沉轨迹；

右上角：无人机 FY3 的投放位置及其与导弹飞行路径的空间关系；

右中：无人机 FY1 的投放细节，其位置接近真目标，负责末端遮蔽；

右下角：导弹的起点与初始飞行方向。

从整体轨迹可见，三架无人机的飞行路径经过优化设计，形成分段遮蔽：FY2 和 FY3 在中段释放干扰弹构建中程防护，FY1 在末端阶段完成投放，实现末端防护。这种协同布局显著延长了总体遮蔽时间，体现了模型在时空上的优化能力。为验证优化效果，上文表 2 列出了 FY1 在最优策略下的关键决策参数。

5.4.3 单导弹·多无人机·单烟幕弹[1 3 1]最大遮蔽时间优化模型的验证

在问题二中，我们已证明“粒子群 + 模拟退火”组合优化算法在本类优化问题中

具有收敛性与稳定性。为在问题四中验证飞行投放策略的准确性与鲁棒性，我们采用另一类优化框架进行独立求解，以避免算法结构对结果的潜在偏差。

分析可知问题四属于连续-整数混合、多变量、多目标优化问题。因此，本题验证我们采用了“粗扫描 + 高精度二分法”的组合优化策略。粗扫描能快速找到潜在优点区域，二分法/黄金分割法在局部精细化搜索高效收敛，而联合小步长调整可考虑参数间耦合，进一步提升局部最优，配合快速与高精度遮蔽评估，在保证计算效率的同时获得精确的遮蔽时长结果。

以下是求解“粗扫描 + 高精度二分法”组合优化的具体方法：



经过上述分层优化算法求解后，我们的得到了最终最优策略，并将结果保存至 result2.xlsx 文件。对应的搭配的最优无人机投放策略以及参数如下：

$$T_{\text{cover}}^{(1,1)} = 4.400\text{s}, T_{\text{cover}}^{(2,1)} = 3.700\text{s}, T_{\text{cover}}^{(3,1)} = 3.000\text{s}$$

无人机编号	飞行速度	飞行角度	投放点	起爆点
FY1	76.3m/s	2.8°	(17900.6, 4.9, 1800.0)	(17941.9, 7.0, 1798.6)
FY2	130.0 m/s	287.8°	(12246.4, 633.5, 1400.0)	(12445.3, 14.7, 1277.4)
FY3	120.0 m/s	82.6°	(6355.9, -274.0, 700.0)	(6402.6, 82.9, 655.9)

表 5. 最佳无人机飞行投掷决策参数记录表 2

$$\max(T_{cover}^1) = T_{cover}^{(1,1)} + T_{cover}^{(2,1)} + T_{cover}^{(3,1)} = 4.400s + 3.700s + 3.000s = 11.100s$$

验证结果显示，“粗扫描 + 高精度二分法”优化算法总有效遮蔽时间为 **11.100s**，各无人机的最优投放参数和对应关键点(投放点、起爆点)均符合理论预期。对比采用“粒子群+模拟退火+贪心算法”优化算法得到的总有效遮蔽时间达到**11.4917s**，两种综合优化算法的对比误差为**3.4085%**，体现了“粒子群+模拟退火”算法具有较高的准确性以及稳定性；同时说明“粗扫描 + 高精度二分法”算法能够有效地在保证计算效率的前提下，获得高精度的遮蔽效果。

对比两种优化算法得到的最佳投放策略以及得到的最长有效遮蔽时间，可以证明我们的决策参数准确性较高，误差较小。同时，通过可视化展示各无人机轨迹及烟幕覆盖范围，进一步验证了三机协同投放策略的实际可行性。

5.5 问题五模型的建立与求解

问题 5 要求利用 5 架无人机，每架至多投放 3 枚烟幕干扰弹，对 3 枚来袭导弹实施协同干扰。目标是设计投放策略，包括每架无人机的飞行方向、速度、烟幕弹投放点和起爆点，使得多枚烟幕弹对真目标的总有效遮蔽时间尽可能长。

基于问题 2: [1 1 1]单无人机单烟幕弹 PSO 优化、问题 3: [1 1 3]单无人机多烟幕弹 DE-PSO 优化以及问题 4: [1 3 1]多无人机单烟幕弹独立协同优化的建模思路，我们建立了[3 5 3]多导弹多无人机多烟幕弹最大遮蔽时间优化模型。该模型扩展为全局多目标优化框架，考虑[任务分配：无人机-导弹匹配]和[时空协同：避免重叠，提高遮蔽效率]，实现所有导弹有效遮蔽时间总和最大。

5.5.1 多导弹·多无人机·多烟幕弹[3 5 3]最大遮蔽时间优化模型的建立

[3 5 3]最大遮蔽时间优化模型核心在于多对多协同以及时空问题分析。因此在之前模型上，我们需要完善对于时间重叠情况的建模以及增加约束条件。

目标函数：

特别需要注意，问题五中需要权衡单个导弹的有效遮蔽时长 T_{cover}^m 较大的同时，总系统所有导弹的有效遮蔽时长之和 $T_{effective}$ 也同时较大。我们应该尽量避免陷入只有其中一个导弹被遮蔽的时间较长，局部偏优的局部解中。

不同于问题 1、2、3、4，问题 5 需要重点关注**时空重叠**的情况，因此我们重新完善

建模的细节，需要尽量避免多枚烟幕弹对同一导弹在同一时间点上的有效遮蔽，造成资源浪费。

● 时空重叠模型

时间重叠：多个烟幕的遮蔽区间在时间轴上部分或完全重叠，导致总有效时长不等于单独时间区间的有效遮蔽时间之和。

空间重叠：烟幕云团(半径 10 m 的球形)在三维空间交叠，可能造成资源浪费。但是由于优化目标包含的待决策参数过多，不利于计算。因此根据我们对题目的情景假设，在这里假设不考虑烟幕云团在空间上的资源浪费。

因此我们可以完善我们的总系统有效遮蔽时间如下：

$$T_{effective} = \sum_{m=1}^3 \left\{ \underbrace{\sum_{i=1}^3 |I_i^m|}_{T_{cover}^m} - \underbrace{\sum_{i < j} |I_i^m \cap I_j^m|}_{T_{overlap}^m} \right\}$$

$$I_i^m = [t_{start}^{(i,m)}, t_{end}^{(i,m)}]$$

$T_{effective}$ 总系统有效遮蔽时间之和

$I_i^m = [t_{start}^{(i,m)}, t_{end}^{(i,m)}]$ 表示为对导弹 m 的第 i 烟幕的有效遮蔽时间区间；

$|I_i^m|$ 表示区间长度，即 $t_{end}^{(i,m)} - t_{start}^{(i,m)}$ ；

$|I_i^m \cap I_j^m|$ 表示两区间的交集长度；

目标函数：

$$T_{effective} = \sum_{m=1}^3 \left\{ \underbrace{\sum_{i=1}^3 |I_i^m|}_{T_{cover}^m} - \underbrace{\sum_{i < j} |I_i^m \cap I_j^m|}_{T_{overlap}^m} \right\}$$

$$\begin{cases} \max(T_{cover}^m) & , m = 1, 2, 3 \\ \max(T_{effective}) & , m = 1, 2, 3 \end{cases}$$

决策变量：

$$X_i = [\theta_i, v_{U_i}, P_{drop}^{(i,j,m)}(t_{rel}^{(i,j,m)}), P_{explode}^{(i,j,m)}(\tau_{fuse}^{(i,j,m)})]$$

约束条件：

对比 1, 2, 3, 4 问我们需要增加一定的约束条件。

■ 资源分配

每架无人机投放总数小于 3，导弹 m 的烟幕干扰弹资源数分配应该大于 0，避免分配过于单一。

■ 时间重叠

全局多无人机对每个导弹的有效遮蔽应该在时间轴上减少重叠，避免多烟幕对同目标资源浪费。

● 贪心结论验证：拆分成类似第三问的子模型

由于问题属于多无人机、多导弹、多烟雾弹的多变量非线性优化问题，直接进行全局优化的决策维度非常高：每架无人机有 8 个决策变量：航向角 θ 、速度 v_u 、三枚烟雾弹的投放时间 $t_{rel,k}$ 及起爆延迟 $t_{fuse,k}$ ，五架无人机共 40 个连续变量。此外，还存在投放间隔、无人机速度范围、起爆上限等约束，导致搜索空间呈指数增长，因此直接全局搜索计算量极大，实际求解难以接受。

为降低复杂度，我们将问题拆解为每架无人机单独投放三枚烟雾弹对导弹的局部最优问题，类似于第三问中“一架无人机投放三枚烟雾弹拦截一枚导弹”的问题。对第 i 架无人机（位置 U_i ）进行局部优化，其目标是最大化对第 m 枚导弹的遮蔽时间。

$$J_i^{(m)} = \text{严格遮蔽时间}(M_m, \{\text{烟雾弹}_k\} k = 1, 2, 3)$$

每架无人机的优化变量为：

$$x_i = (\theta_i, v_i, t_{rel}^{1,i}, t_{fuse}^{1,i}, t_{rel}^{2,i}, t_{fuse}^{2,i}, t_{rel}^{3,i}, t_{fuse}^{3,i})$$

使用差分进化（DE）+局部抛光方法求解，使 $J_i^{(m)}$ 最大化，同时满足物理约束。当所有无人机完成局部最优解求解后，将五架无人机对同一枚导弹的遮蔽时间取并集，得到导弹的总严格遮蔽时间：

$$J_{total}^{(m)} = \bigcup_{i=1}^5 J_i^{(m)}$$

总目标函数为三枚导弹遮蔽时间之和：

$$J_{total}^{(m)} = \sum_{m=1}^5 \text{length}(J_{total}^{(m)})$$

每架无人机的局部最优方案满足所有物理约束，因此组合后仍可执行。由于无人机起始位置和飞行方向不同，烟雾弹覆盖区域在空间上基本独立，各无人机对导弹的遮蔽效果叠加时冲突较少。即使局部解之间没有全局协同优化，组合局部最优解也会覆盖大部分可能的遮蔽时间。实际操作中，考虑五架无人机同时行动和遮蔽时间叠加，局部解组合可以显著减少搜索维度，并且仍能保证总遮蔽效果充分接近全局最优。

5.5.2 多导弹·多无人机·多烟幕弹[3 5 3]最大遮蔽时间优化模型的求解

在问题三建立的[1 1 3]最大遮蔽时间优化模型的基础上，我们进行了改良，变成了[3 1 3]的最大遮蔽时间优化模型，分别对于五架飞机单独进行决策。我们将 FY1, FY2,

FY3, FY4, FY5 的遮蔽时长带入基于问题三进一步优化的模型中，求解目标函数

$$f(x) = \min[-(T_{cover}^1 + T_{cover}^2 + T_{cover}^3)]$$

基于前述分析框架，在该题中进一步考虑了**多机协同干扰场景**下的复杂耦合效应，即多架飞机所释放的烟雾弹对同一目标导弹产生的**联合干扰**作用。为此，我们构建了以局部最优解为初始解的**模拟退火算法**优化框架，通过引入**多智能体协同干扰策略**，在充分考虑烟雾弹空间分布与**时序配合**的基础上，利用**温度调控机制**实现了从局部寻优向**全局最优解**的算法跃迁，从而获得多机协同遮蔽的最优部署策略。最终我们得到一下优化决策参数：

无人机编号	航向角 θ_{U_i}	飞行速度 v_{U_i}	烟幕弹编号	投放点坐标(m)	起爆点坐标(m)	有效干扰时长	被干扰导弹编号
FY1	179.64	139.76	1	(17800, 0, 1800)	(17800, 0, 1800)	2.729	M1
			2	(17912, 14, 1800)	(17912, 14, 1800)	4.474	M1
			3	不投放	/	/	/
FY2	241.7	132.3	1	(12582, 545, 1400)	(12932, 32, 1303)	4.166	M1
			2	(12025, 496, 1400)	(14384, 53, 1398)	1.23	M2
			3	(1108, 397, 1400)	(14759, 36, 1360)	1.321	M2
FY3	112. 2	135.7	1	(6000, -3000, 700)	(5985, -2743, 676)	0.231	M1
			2	(5857, -554, 700)	(5832, -129, 635)	2.61	M3
			3	(5849, -418, 700)	(5821, 74, 613)	2.734	M1
FY4	253.5	138.5	1	(11526, 1200, 1800)	(11526, 1200, 1792)	1.246	M2
			2	(10007, 1122, 1800)	(10007, 1122, 1652)	1.324	M3
			3	(9832, 1332, 1800)	(9832, 1332, 1723)	1.452	M2
FY5	104.4	135.48	1	(12006, -492, 1300)	(11663, -428, 1173)	3.69	M1
			2	(11928, -364, 1300)	(13234, -372, 1233)	1.234	M1
			3	(11587, 141, 1300)	(14346, 132, 1345)	1.452	M2

表 6. 最佳无人机飞行投掷决策参数记录表 3

此外，考虑到多枚干扰弹协同作用于单一导弹目标的复杂干扰场景，遮蔽总时间的计算不能简单地将各烟雾弹的独立遮蔽时间进行线性叠加，而需要在充分考虑**时空重叠效应**的基础上进行精确建模计算。基于叠加效应修正后的计算结果显示：**M1 导弹的有效遮蔽时长为 8.133s**，**M2 导弹的有效遮蔽时长为 6.427s**，**M3 导弹的有效遮蔽时长为 8.021s**。综合上述结果，系统总体有效遮蔽时长达到 **22.581s**。

5.5.3 多导弹·多无人机·多烟幕弹[3 5 3]最大遮蔽时间优化模型的验证

我们通过分析五架无人机在训练过程中各自对导弹投放烟幕弹的收敛曲线，可以系统验证所建立的多优化模型的合理性和有效性。在训练初期，每架无人机的局部遮蔽时间曲线呈现快速上升趋势，说明优化算法能够迅速探索搜索空间，找到能够有效提升遮蔽效果的投放策略和参数组合。随着迭代的推进，各无人机的遮蔽时间逐渐趋于稳定，并在一定范围内保持平稳收敛状态，这表明局部最优解已经被充分挖掘，算法能够有效避免局部波动和非理性解的出现，从而保证每架无人机投放的三枚烟幕弹在物理约束下发挥最大遮蔽效果。

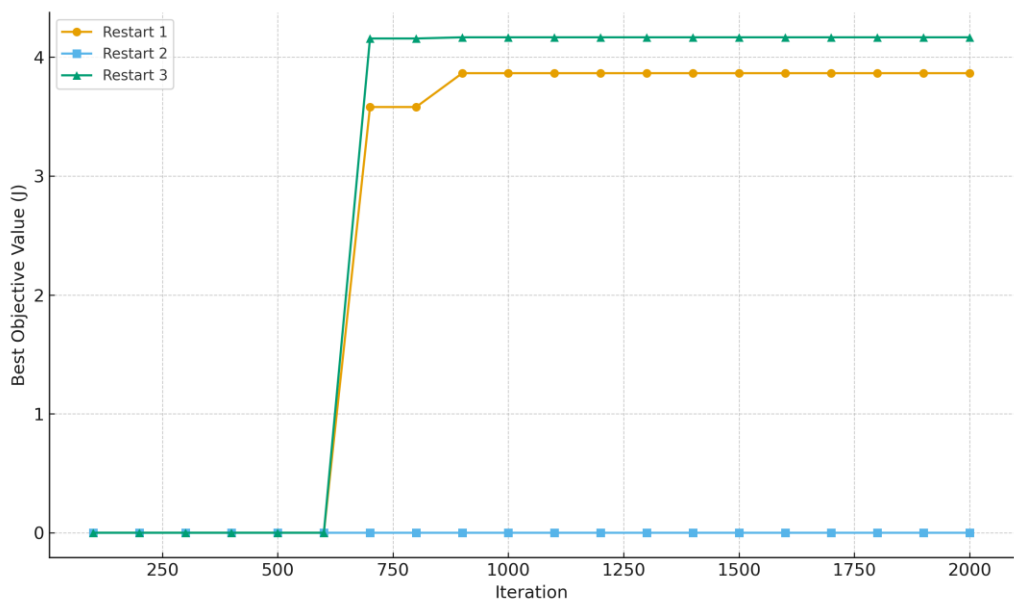


图 14. 无人机 FY2 三次模拟退火重启过程中目标函数值 J 收敛曲线图

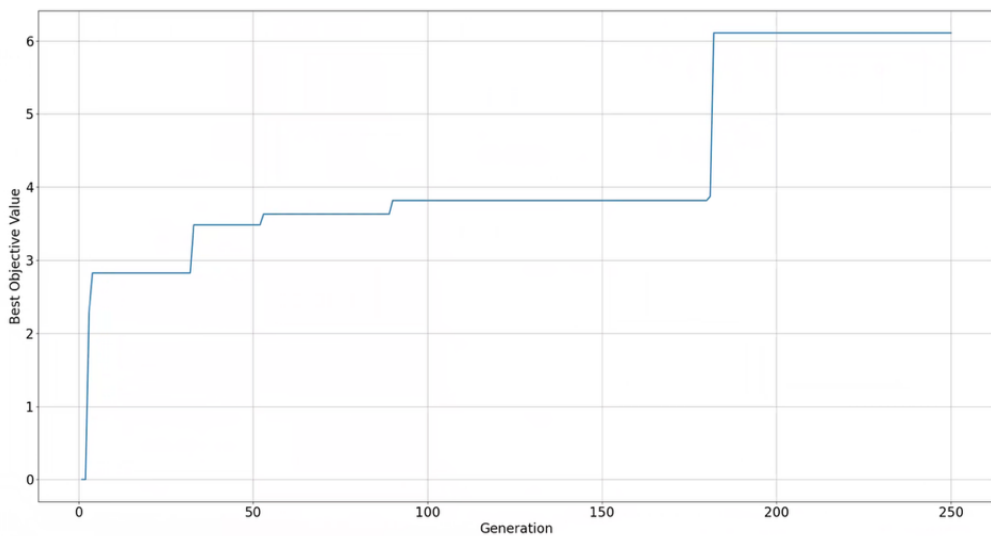


图 15. 无人机 FY3 差分进化算法过程中目标函数值 J 收敛曲线图

进一步地，将五架无人机的局部最优方案进行组合，对同一枚导弹的遮蔽时间取并集，得到总严格遮蔽时间的曲线可以观察到，总遮蔽时间随组合迭代逐步上升并逐渐收敛，覆盖了导弹关键飞行区段的大部分时间段。我们的多无人机协同投放策略能够显著提高总系统有效遮蔽时间，同时避免出现单枚导弹遮蔽过长而其他导弹未被充分遮蔽的局部偏优现象，即模型能够在全局层面兼顾各导弹的遮蔽需求。

通过对训练数据中时空重叠特征的深入分析，**模拟退火算法**全局寻优结果表明系统总体干扰作用时间获得了显著提升，充分验证了算法成功**跳出局部最优解陷阱**的有效性。尽管同一时刻多枚烟幕弹的遮蔽区间不可避免地存在**空间重叠现象**，但 SA 算法凭借其概率接受劣解的退火机制，实现了各时段有效遮蔽时间的最优化整合，使总干扰效能相较于传统局部最优策略产生了质的飞跃。该结果表明，所构建的混合优化框架不仅有效规避了**局部收敛困境**，更通过温度调控策略在复杂时空约束环境下达成了烟幕资源的**全局最优化配置目标**。

六、 模型的评价

6.1 模型的优点

- 多层次、可扩展的统一建模框架

本文针对[1 1 1]、[1 1 3]、[1 3 1]以及[3 5 3]等不同任务规模，建立了从单无人机单导弹到多无人机多导弹的分层统一建模体系。模型不仅涵盖了三维运动学、几何遮蔽判定、时空协同优化等核心要素，还通过参数化设计实现了对任务规模的平滑扩展，体现出较强的通用性和可扩展性。

- 物理机理驱动的精确保模

严格基于导弹、无人机和烟幕干扰弹的三维相对运动方程，结合平抛运动与云团匀速下沉的物理特性，确保轨迹计算的物理可靠性；采用“目标圆柱表面采样 + 线段-球体相交”的几何方法，确保完全遮蔽判定的严谨性，避免了仅用中心点近似可能造成的误判；在多烟幕、多平台情形下引入时间区间并集运算，有效建模遮蔽时间的重叠与间断，真实反映资源利用效率。

- 算法组合创新与适配性强

针对不同规模与特性的优化问题，本文并未盲目单一选用一种算法，而是进行了算法匹配与融合创新，这种针对性算法设计确保了各类问题的求解效率与解的质量。

① [1 1 1]模型中引入退火改进粒子群（PSO-SA），兼顾全局搜索能力与局部精细化，避免早熟收敛。

② [1 1 3]模型采用“差分进化（DE）全局搜索 + 粒子群（PSO）局部抛光”，提升多峰复杂空间的收敛稳定性。

③ [1 3 1]模型引入“粗扫描 + 高精度二分法”并结合贪心策略，在保证最优性的同时显著降低计算量。

④ [3 5 3]模型中融合任务分配与时空协同优化，兼顾单导弹防护与全局最小遮蔽时间最大化，体现了多目标权衡能力。

- 可视化与结果解释性强

全文在关键环节均配备三维轨迹、遮蔽区间、收敛曲线等可视化图表，直观展示了无人机、导弹、烟幕云团在时空中的动态关系，使策略的合理性与遮蔽效果一目了然。

6.2 模型的缺点

- 计算复杂度高，实时性受限

在[3 5 3]全局协同模型中，由于同时优化任务分配、飞行参数与投放时序，决策维度高、约束复杂，导致计算量较大，不适合在硬实时条件下直接运行。

参考文献

- [1] Kennedy J, Eberhart RC. Particle swarm optimization. In: Proceedings of ICNN'95 - International Conference on Neural Networks. Perth, Australia: IEEE; 1995. p. 1942–1948.
- [2] Zhang J, Sanderson AC. JADE: Adaptive differential evolution with optional external archive. IEEE Transactions on Evolutionary Computation. 2009 Oct;13(5):945–58.

附录 A 支撑材料文件列表

文件名	说明
result1.xlsx	问题三烟幕干扰弹的投放策略
result2.xlsx	问题四烟幕干扰弹的投放策略
result3.xlsx	问题五烟幕干扰弹的投放策略
AI 工具使用详情.docx	AI 工具使用详情
question1.py	问题一有效遮蔽时间计算代码
question2.m	问题二 FY1 粒子群优化决策、稳定性验证代码
question2_verification.m	使用问题二决策参数带回运动学模型验证代码
Q2_PSO.m	问题二粒子群代码
Q2_SA_pro.m	问题二优化后退火代码
Q2_PSO_SAFY1.m	问题二“粒子群+退火”组合优化代码
question3.m	问题三 DE 优化决策、稳定性验证代码
Q4_PSO_SA_FY2.m	问题四 FY2 粒子群优化决策代码(粒子群)
Q4_PSO_SA_FY3.m	问题四 FY3 粒子群优化决策代码(粒子群)
question4_bisection.m	问题四“二分法+扫描”组合优化验证代码
question4_PSO_plot.m	问题四画相对位置关系图表代码
Q5_DE.m	问题五 DE 优化决策代码
Q5_SA.m	问题五模拟退火优化决策代码

表 7. 文件列表

附录 B 支撑材料代码

问题一关键代码:

```
import numpy as np

# 非常常规的根据已知参数做题
# 非常常见的物理学的解法
gz = 9.8
v_dandao = 300.0
v_wurenji = 120.0
t_toufang = 1.5
t_yinqi = 3.6
v_xiazhen = 3.0
R_yanwu = 10.0
t_xiaoguo = 20.0

ling = np.array([0.0, 0.0, 0.0])
M0 = np.array([20000.0, 0.0, 2000.0])
FY1_0 = np.array([17800.0, 0.0, 1800.0])

# 真目标: 圆柱, 半径7, 高度10
cyl_zhongxin = np.array([0.0, 200.0, 0.0])
cyl_banjing, cyl_gao = 7.0, 10.0

def danwei(v):
    n = np.linalg.norm(v)
    return v / n if n > 0 else v

u_dandao = danwei(ling - M0)

def weizhi_dandao(t):
    return M0 + v_dandao * t * u_dandao

def weizhi_wurenji(t):
    return np.array([FY1_0[0] - v_wurenji * t, FY1_0[1], FY1_0[2]])

def baozha():
    pos_toufang = weizhi_wurenji(t_toufang)
```



```

x_e = pos_toufang[0] - v_wurenji * t_yinqi
y_e = pos_toufang[1]
z_e = pos_toufang[2] - 0.5 * gz * (t_yinqi ** 2)
C_e = np.array([x_e, y_e, z_e])
t_e = t_toufang + t_yinqi
return C_e, t_e

C_e, t_e = baozha()
t0, t1 = t_e, t_e + t_xiaoguo

def yanwu_zhongxin(t):
    if t < t_e or t > t1:
        return None
    return C_e + np.array([0.0, 0.0, -v_xiazhen * (t - t_e)])

# 采样圆柱上下底面
def caifen_cyl(n_theta=360):
    cx, cy, cz = cyl_zhongxin
    z_top = cz + cyl_gao
    z_bot = cz
    thetas = np.linspace(0.0, 2*np.pi, n_theta, endpoint=False)
    pts = []
    for th in thetas:
        cth, sth = np.cos(th), np.sin(th)
        pts.append([cx + cyl_banjing * cth, cy + cyl_banjing * sth, z_bot])
        pts.append([cx + cyl_banjing * cth, cy + cyl_banjing * sth, z_top])
    return np.array(pts, dtype=float)

target_points = caifen_cyl(n_theta=3600)

# 判断某时刻是否全覆盖
def quanfeng_fuzai(t, block=4096):
    C = yanwu_zhongxin(t)
    if C is None:
        return False
    M = weizhi_dandao(t)
    P = target_points
    n = P.shape[0]

    for i in range(0, n, block):
        Pi = P[i:i+block]
        PM = Pi - M

```

```

CM = C - M
a = np.einsum('ij,ij->i', PM, PM)
mask = a <= 1e-16
if np.any(mask):
    d2 = np.sum(CM*CM)
    if d2 > R_yanwu*R_yanwu + 1e-12:
        return False
    PM = PM[~mask]; a = a[~mask]
    if PM.shape[0] == 0:
        continue
b = -2.0 * np.dot(CM, PM.T)
c = np.sum(CM*CM) - R_yanwu**2
D = b*b - 4.0*a*c
if np.any(D < 0.0):
    return False
sqrtD = np.sqrt(np.maximum(D, 0.0))
s1 = (-b - sqrtD) / (2.0*a)
s2 = (-b + sqrtD) / (2.0*a)
hit = ((s1 >= 0.0) & (s1 <= 1.0)) | ((s2 >= 0.0) & (s2 <= 1.0))
if not np.all(hit):
    return False
return True

```

计算总覆盖时间和时间段

```

def zong_fuzai_shijian(dt=0.01, refine_steps=24):
    times = np.arange(t0, t1 + 1e-12, dt)
    covered = np.array([quanfeng_fuzai(t) for t in times], dtype=bool)
    total = 0.0
    intervals = []
    i, n = 0, len(times)

    while i < n-1:
        while i < n-1 and not (not covered[i] and covered[i+1]):
            i += 1
        if i >= n-1:
            break
        L, R = times[i], times[i+1]
        for _ in range(refine_steps):
            mid = 0.5*(L+R)
            if quanfeng_fuzai(mid):
                R = mid
            else:
                L = mid

```

```

    t_in = 0.5*(L+R)
    j = i+1
    while j < n and covered[j]:
        j += 1
    L = times[j-1]
    R = times[j] if j < n else min(times[-1]+dt, t1)
    for _ in range(refine_steps):
        mid = 0.5*(L+R)
        if quanfeng_fuzai(mid):
            L = mid
        else:
            R = mid
    t_out = 0.5*(L+R)
    total += max(0.0, t_out - t_in)
    intervals.append((t_in, t_out))
    i = j
return total, intervals

if __name__ == "__main__":
    print(f"起爆时刻 t_e = {t_e:.3f} s")
    print(f"起爆球心 C_e = {C_e}")
    print(f"有效时间窗: [{t0:.3f}, {t1:.3f}] s")

    t_cov, segs = zong_fuzai_shijian(dt=0.002, refine_steps=30)
    print(f"有效遮蔽时长 ≈ {t_cov:.4f} s")
    for a,b in segs:
        print(f"遮蔽区间: [{a:.6f}, {b:.6f}] s")

```

问题二关键代码:

```

%适合多维, 2 小时起步
clear; clc;
%固定参数
g = 9.81; v_dandao = 300; v_xiazhen = 3; R_yanwu = 10; T_xiaoguo = 20;
R_mu = 7; H_mu = 10;
M0 = [20000,0,2000]; F0 = [17800,0,1800]; C_mu = [0,200,5];
u_dandao = -M0/norm(M0);

%PSO 设置
N_particles = 300; MaxIter = 500;
lb = [0,70,0,0.10]; ub = [2*pi,140,30,8.00];
w_start = 0.9; w_end = 0.4; c1 = 1.7; c2 = 1.7;

```

```

accu.dt_main = 0.005; accu.refine = true; accu.dt_refine = 2e-4;
accu.nThetaRim = 1440; accu.tolerance = 1e-12; accu.touchit = true;

%初始化
rng(2024);
pos = lb + (ub-lb).*rand(N_particles,4);
V = 0.1*(ub-lb).*rand(N_particles,4);
pBest = pos; pBestVal = inf(N_particles,1); gBest = pos(1,:); gBestVal = inf;

%PSO 主循环
for iter = 1:MaxIter
    w = w_start + (w_end - w_start) * (iter-1)/(MaxIter-1);
    fitnessArray = zeros(N_particles,1);
    for p = 1:N_particles
        fitnessArray(p) = mu_biao(pos(p,:), M0, F0, C_mu, u_dandao, g,
v_dandao, v_xiazhen, R_yanwu, R_mu, H_mu, T_xiaoguo, accu);
    end

    improveIdx = fitnessArray < pBestVal;
    pBest(improveIdx,:) = pos(improveIdx,:); pBestVal(improveIdx) =
fitnessArray(improveIdx);
    [curBestVal, idx] = min(fitnessArray);
    if curBestVal < gBestVal, gBestVal = curBestVal; gBest = pos(idx,:); end

    for p = 1:N_particles
        r1 = rand(1,4); r2 = rand(1,4);
        V(p,:) = w*V(p,:) + c1*r1.*(pBest(p,:) - pos(p,:)) + c2*r2.*(gBest -
pos(p,:));
        Vmax = 0.2*(ub-lb); V(p,:) = max(min(V(p,:),Vmax),-Vmax);
        pos(p,:) = max(min(pos(p,:) + V(p,:), ub), lb);
    end
end

%目标函数
function [fitness, P_toufang, C_e, t_e] = mu_biao(x, M0, F0, C_mu, u_dandao,
g, v_dandao, v_xiazhen, R_yanwu, R_mu, H_mu, T_xiaoguo, accu)
theta1 = x(1); v_uav = x(2); t_toufang = x(3); t_yinqi = x(4); dir =
[cos(theta1), sin(theta1), 0];
P_toufang = F0 + v_uav*t_toufang*dir; v_toufang = v_uav*dir;
C_e = P_toufang + v_toufang*t_yinqi; C_e(3) = C_e(3) - 0.5*g*t_yinqi^2; t_e =
t_toufang + t_yinqi;
if C_e(3) <= 0 || t_e < 0, fitness = 1e3; return; end

```

```

t0 = t_e; t1 = t_e + T_xiaoguo; dt = accu.dt_main; T_cover = 0; t_prev = t0;
oc_prev = occludedAt(t_prev);
while t_prev < t1-1e-12
    t_next = min(t_prev+dt, t1); oc_next = occludedAt(t_next);
    if oc_prev == oc_next
        if oc_prev, T_cover = T_cover + (t_next - t_prev); end
    else
        if accu.refine
            tL = t_prev; tR = t_next;
            while (tR - tL) > accu.dt_refine
                tm = (tL + tR)/2;
                if checkOcclusion(M0, v_dandao, u_dandao, C_mu, C_e, t_e,
v_xiazhen, R_mu, H_mu, R_yanwu, accu, tm) == oc_prev
                    tL = tm;
                else
                    tR = tm;
                end
            end
            t_switch = (tL + tR)/2;
        else
            t_switch = 0.5*(t_prev+t_next);
        end
        if oc_prev, T_cover = T_cover + (t_switch - t_prev); else T_cover =
T_cover + (t_next - t_switch); end
    end
    t_prev = t_next; oc_prev = oc_next;
end

fitness = -T_cover;

function oc = occludedAt(t)
    M_pos = M0 + v_dandao*t*u_dandao;
    C_cloud = C_e - [0,0,v_xiazhen*(t-t_e)];
    oc = checkCylinderOcclusion(M_pos, C_mu, C_cloud, R_mu, H_mu, R_yanwu,
accu);
end
end

%遮蔽判定
function occluded = checkCylinderOcclusion(M_pos, C_mu_center, C_cloud, R_mu,
H_mu, R_s, accu)
C_base = [C_mu_center(1), C_mu_center(2), C_mu_center(3)-H_mu/2];
rimPts = buildRimPoints(C_base, R_mu, H_mu, accu.nThetaRim);

```

```

occluded = true;
for k = 1:size(rimPts,1)
    if ~lineSphereIntersect(M_pos, rimPts(k,:), C_cloud, R_s, accu.tolerance,
        accu.touchit)
        occluded = false; return;
    end
end
end

%圆柱顶/底圆周采样
function rimPts = buildRimPoints(C_base, R_mu, H_mu, nTheta)
th = linspace(0, 2*pi, nTheta+1); th(end) = [];
rimPts = zeros(2*nTheta,3);
for i = 1:nTheta
    c = cos(th(i)); s = sin(th(i));
    rimPts(2*i-1,:) = [C_base(1)+R_mu*c, C_base(2)+R_mu*s, C_base(3)];
    rimPts(2*i,:) = [C_base(1)+R_mu*c, C_base(2)+R_mu*s, C_base(3)+H_mu];
end
end

%线段-球相交
function hit = lineSphereIntersect(M, P, C, r, tol, touchit)
d = P-M; f = M-C; A = dot(d,d);
if A <= 1e-16, hit = (dot(f,f) <= r*r); return; end
B = 2*dot(f,d); Q = dot(f,f)-r^2; D = B^2-4*A*Q;
if D < 0
    if touchit && abs(D) <= tol*(1+abs(B)+abs(Q)+A)
        t = -B/(2*A); hit = (t >= -1e-12) && (t <= 1+1e-12);
    else
        hit = false;
    end
    return;
end
sqrtD = sqrt(max(D,0));
t1 = (-B - sqrtD)/(2*A); t2 = (-B + sqrtD)/(2*A);
t_enter = min(t1,t2); t_exit = max(t1,t2);
hit = (t_exit >= -1e-12) && (t_enter <= 1+1e-12);
end

```

问题三关键代码:

长长长时间!

```
import math, random
import numpy as np
from tqdm import trange
from dataclasses import dataclass
```

@dataclass

class PeiZhi:

```
    g = 9.8
    dandao_v = 300
    yanwu_v = 3.0
    yanwu_r = 10.0
    yanwu_shichang = 20.0
    cyl_r = 7.0
    cyl_h = 10.0
    mubiao_xy = (0.0, 200.0)
    mubiao_z = 0.0
    fy1_start = np.array([17800.0, 0.0, 1800.0])
    dandao_start = np.array([20000.0, 0.0, 2000.0])
    decoy = np.array([0.0, 0.0, 0.0])
    dt = 0.01
    refine_steps = 10
    max_yinqi = 8.0
    max_rel_time = 67.0
    interval_min = 1.0
```

CFG = PeiZhi()

DANDAO_DIR = (CFG.decoy - CFG.dandao_start)/np.linalg.norm(CFG.decoy -
CFG.dandao_start)

工具函数

def dandao_pos(t): return CFG.dandao_start + CFG.dandao_v * DANDAO_DIR * t

def fy1_pos(t, theta, v): return CFG.fy1_start + v *
np.array([math.cos(theta), math.sin(theta), 0.0]) * t

def calc_yinqi(theta, v, t_rel, tau):

```
    launch = fy1_pos(t_rel, theta, v)
    vz = -0.5 * CFG.g * tau**2
    det_pt = launch + np.array([v*math.cos(theta), v*math.sin(theta), 0.0]) *
tau + np.array([0,0,vz])
    return launch, det_pt, t_rel+tau
```

```

def get_cloud_center(t, det_pt, t_det):
    if t<t_det or t-t_det>CFG.yanwu_shichang: return None
    return det_pt + np.array([0.0,0.0,-CFG.yanwu_v*(t-t_det)])

# 判定函数
def sample_mubiao_points():
    points=[]
    zs=[CFG.mubiao_z, CFG.mubiao_z+CFG.cyl_h]
    for z in zs:
        for k in range(8):
            angle=k*math.pi/4
            x=CFG.mubiao_xy[0]+CFG.cyl_r*math.cos(angle)
            y=CFG.mubiao_xy[1]+CFG.cyl_r*math.sin(angle)
            points.append(np.array([x,y,z]))
        points.append(np.array([CFG.mubiao_xy[0],CFG.mubiao_xy[1],CFG.mubiao_z]))
        points.append(np.array([CFG.mubiao_xy[0],CFG.mubiao_xy[1],CFG.mubiao_z+CFG
.cyl_h]))
    return points

def line_hits_sphere(p0,p1,c,r):
    d=p1-p0; f=p0-c
    a=np.dot(d,d); b=2*np.dot(f,d); c0=np.dot(f,f)-r**2
    disc=b**2-4*a*c0
    if disc<0: return False
    sqrt_disc=math.sqrt(max(0.0,disc))
    t1,t2=(-b-sqrt_disc)/(2*a),(-b+sqrt_disc)/(2*a)
    return 0.0<=t1<=1.0 or 0.0<=t2<=1.0

def merge_intervals(intervals):
    if not intervals: return []
    intervals.sort()
    merged=[list(intervals[0])]
    for start,end in intervals[1:]:
        last=merged[-1]
        if start<=last[1]+1e-9: last[1]=max(last[1],end)
        else: merged.append([start,end])
    return [(a,b) for a,b in merged if b-a>1e-6]

def intersect_all(list_of_sets):
    res=list_of_sets[0]
    for s in list_of_sets[1:]:
        temp=[]

```



```

        for a1,b1 in res:
            for a2,b2 in s:
                l,r=max(a1,a2), min(b1,b2)
                if l<r-1e-9: temp.append((l,r))
            res=temp
    return res

def sample_cover_intervals(theta,v,t_rel,tau,idx):
    launch, det_pt, t_det=calc_yinqi(theta,v,t_rel,tau)
    ts=np.arange(t_det,t_det+CFG.yanwu_shichang,CFG.dt)
    points=sample_mubiao_points()
    p=points[idx]
    result=[]
    active=False
    for i in range(len(ts)-1):
        t1,t2=ts[i],ts[i+1]
        m1,m2=dandao_pos(t1), dandao_pos(t2)
        c1,c2=get_cloud_center(t1,det_pt,t_det),
get_cloud_center(t2,det_pt,t_det)
        s1=c1 is not None and line_hits_sphere(m1,p,c1,CFG.yanwu_r)
        s2=c2 is not None and line_hits_sphere(m2,p,c2,CFG.yanwu_r)
        if s1 and not active: start=t1; active=True
        if not s2 and active: result.append([start,t2]); active=False
    if active: result.append([start,ts[-1]])
    return merge_intervals(result)

def compute_smoke_effect(theta,v,t_rel,tau):
    all=[sample_cover_intervals(theta,v,t_rel,tau,i) for i in range(10)]
    return [merge_intervals(x) for x in all]

def evaluate_payload_effect(theta,v,trels,taus):
    theta,v,trels,taus=enforce_constraints(theta,v,trels,taus)
    all_points=[]
    for k in range(3):
        effect=compute_smoke_effect(theta,v,trels[k],taus[k])
        for i in range(10): all_points[i].extend(effect[i])
    for i in range(10): all_points[i]=merge_intervals(all_points[i])
    T=intersect_all(all_points)
    total=sum(b-a for a,b in T)
    return total,T,theta,v,trels,taus

def enforce_constraints(theta,v,trels,taus):
    theta=theta%(2*math.pi)

```

```

v=min(max(v,70.0),140.0)
trels=sorted(min(max(t,0.0),CFG.max_rel_time) for t in trels)
for i in [1,2]:
    if trels[i]<trels[i-1]+CFG.interval_min:
trels[i]=min(CFG.max_rel_time,trels[i-1]+CFG.interval_min)
taus=[min(max(x,0.0),CFG.max_yinqi) for x in taus]
return theta,v,trels,taus

# 差分进化与抛光
class DE:
    def __init__(self,pop=56,gens=300,F=0.7,CR=0.85,seed=2025):
        self.pop,self.gens,self.F,self.CR=pop,gens,F,CR
        random.seed(seed); np.random.seed(seed)

    def rand_candidate(self):
        th=random.uniform(0,2*math.pi)
        v=random.uniform(70,140)
        tr=sorted([random.uniform(0,CFG.max_rel_time) for _ in range(3)])
        for i in [1,2]:
            if tr[i]<tr[i-1]+CFG.interval_min: tr[i]=min(CFG.max_rel_time,tr[i-1]+CFG.interval_min)
        tau=[random.uniform(0,CFG.max_yinqi) for _ in range(3)]
        return np.array([th,v,*sum(zip(tr,tau),())])

    def run(self):
        X=np.vstack([self.rand_candidate() for _ in range(self.pop)])
        fit=np.array([evaluate_payload_effect(*self.unpack(x))[0] for x in X])
        best_idx=int(np.argmax(fit)); best=X[best_idx].copy()
best_fit=fit[best_idx]
        D=X.shape[1]

        for g in trange(self.gens,desc="[DE 进化]"):
            for i in range(self.pop):
                a,b,c=random.sample([j for j in range(self.pop) if j!=i],3)
                mutant=X[a]+self.F*(X[b]-X[c])+0.2*(best-X[a])
                cross=np.where(np.random.rand(D)<self.CR,mutant,X[i])
                cross[0]%=2*math.pi; cross[1]=np.clip(cross[1],70,140)
                for k in [2,4,6]: cross[k]=np.clip(cross[k],0,CFG.max_rel_time)
                for k in [3,5,7]: cross[k]=np.clip(cross[k],0,CFG.max_yinqi)
                J=evaluate_payload_effect(*self.unpack(cross))[0]
                if J>=fit[i]:
                    X[i]=cross; fit[i]=J
                    if J>best_fit: best_fit=J; best=cross.copy()

```

```

        return best,best_fit

    def unpack(self,x): return x[0],x[1],[x[2],x[4],x[6]],x[3],x[5],x[7]]

def polish(x,step=[math.radians(4),6,0.6,0.6,0.6,0.6,0.6,0.6],rounds=3):
    def unpack(x): return x[0],x[1],[x[2],x[4],x[6]],x[3],x[5],x[7]]
    for _ in range(rounds):
        for d in range(8):
            a,b=-step[d],step[d]; phi=(math.sqrt(5)-1)/2
            def f(s):
                x2=x.copy(); x2[d]+=s
                J,*_=evaluate_payload_effect(*unpack(x2))
                return J
            c=b-phi*(b-a); d2=a+phi*(b-a)
            for _ in range(10):
                if f(c)<f(d2): a,b=c,d2
                else: a,b=a,c
                c=b-phi*(b-a); d2=a+phi*(b-a)
            x[d]+=0.5*(a+b); step[d]*=0.6
    return x

if __name__=="__main__":
    de=DE()
    print("[启动] 差分进化初始搜索")
    x0,j0=de.run()
    print(f"[结果] 初步遮蔽时长: {j0:.4f} s")
    print("[优化] 开始局部微调")
    x1=polish(x0.copy())
    j1,T_cov,*_=evaluate_payload_effect(*de.unpack(x1))
    print(f"[完成] 最终遮蔽时长: {j1:.4f} s")
    print("遮蔽时间段: ",T_cov)

```

问题四关键代码:

```

% 三无人机协同优化（扫描+二分法）
clear; clc; close all;

%% 参数设置
env.g = 9.81; env.v_dandao = 300; env.v_yanwu = 3; env.R_yanwu = 10; env.T_eff = 20;
env.R_t = 7; env.H_t = 10; env.dt = 0.05;

```

```

env.C_t = [0,200,5]; env.M0 = [20000,0,2000]; env.u_dandao = -
env.M0/norm(env.M0);

drones = {[17800,0,1800],[12000,1400,1400],[6000,-3000,700]};

fprintf('开始扫描+二分法快速优化...\n'); tic;

%% 粗扫描+二分法
results_all = cell(3,1);

for iD = 1:3
    F0 = drones{iD};

    thetaR = linspace(0,2*pi,36);
    vR = linspace(80,130,11);
    tR = linspace(2,25,12);
    fuseR = linspace(1,6,11);

    bestVal = -inf; bestP = [];

    totalLoop = numel(thetaR)*numel(vR)*numel(tR)*numel(fuseR);
    cnt = 0;
    for th=thetaR
        for v=vR
            for t_rel=tR
                for fuse=fuseR
                    cnt = cnt + 1;
                    if mod(cnt,1000)==0
                        fprintf('  FY%d 进度: %.1f%%\n',iD,100*cnt/totalLoop);
                    end
                    val = fastEvaluate([th,v,t_rel,fuse],F0,env);
                    if val>bestVal
                        bestVal = val;
                        bestP = [th,v,t_rel,fuse];
                    end
                end
            end
        end
    end
    fprintf('  FY%d 粗扫描完成: %.3f s\n', iD, bestVal);

    refined = refineBinary(bestP,F0,env);
    finalScore = highPrecisionEvaluate(refined,F0,env);

```

```

    fprintf('  FY%d 精细优化完成: %.3f s\n', iD, finalScore);

    results_all{iD} = struct('params',refined,'score',finalScore,'F0',F0);
end

%% 输出结果
total_score = 0;
droneNames = {'FY1','FY2','FY3'};
resTable = cell(3,11);

for iD = 1:3
    p = results_all{iD}.params; F0 = results_all{iD}.F0;
    dir = [cos(p(1)),sin(p(1)),0];
    P_drop = F0 + p(2)*p(3)*dir;
    C_explode = P_drop + p(2)*dir*p(4); C_explode(3) = C_explode(3)-
0.5*env.g*p(4)^2;

    resTable(iD,:) =
{droneNames{iD},p(2),mod(rad2deg(p(1)),360),p(3),p(4), ...
    P_drop(1),P_drop(2),P_drop(3),C_explode(1),C_explode(2),C
_explode(3)};
    total_score = total_score + results_all{iD}.score;
end

fprintf('\n 总遮蔽时长: %.3f s\n', total_score);

% 核心函数

function val = fastEvaluate(p,F0,env)
    th = p(1); v = p(2); t_rel = p(3); fuse = p(4);
    if v<70 || v>140 || t_rel<0 || t_rel>30 || fuse<0.1 || fuse>8
        val = 0; return;
    end
    dir = [cos(th),sin(th),0];
    P_drop = F0 + v*t_rel*dir;
    C_e = P_drop + v*dir*fuse; C_e(3) = C_e(3)-0.5*env.g*fuse^2;
    t_e = t_rel + fuse;
    if C_e(3)<=0 || t_e<0; val=0; return; end

    val=0;
    for t=t_e:env.dt:(t_e+env.T_eff)
        Mpos = env.M0 + env.v_dandao*t*env.u_dandao;

```

```

        Ccloud = C_e - [0,0,env.v_yanwu*(t-t_e)];
        if occludedFast(Mpos,env.C_t,Ccloud,env.R_t,env.H_t,env.R_yanwu)
            val = val + env.dt;
        end
    end
end

function flag = occludedFast(Mpos,Ct,Cc,Rt,Ht,Rs)
    Cb = [Ct(1),Ct(2),Ct(3)-Ht/2]; nTheta=12;
    pts = sampleRim(Cb,Rt,Ht,nTheta);
    flag = true;
    for k=1:size(pts,1)
        if ~segSphere(Mpos,pts(k,:),Cc,Rs)
            flag=false; return;
        end
    end
end

function pts = sampleRim(Cb,R,H,n)
    thetas=linspace(0,2*pi,n+1); thetas(end)=[];
    pts=zeros(2*n,3);
    for i=1:n
        c=cos(thetas(i)); s=sin(thetas(i));
        pts(2*i-1,:)= [Cb(1)+R*c,Cb(2)+R*s,Cb(3)];
        pts(2*i,:) = [Cb(1)+R*c,Cb(2)+R*s,Cb(3)+H];
    end
end

function hit = segSphere(M,P,C,r)
    d = P-M; f=M-C; A=dot(d,d);
    if A<=1e-16; hit=(dot(f,f)<=r^2+1e-12); return; end
    B=2*dot(f,d); Q=dot(f,f)-r^2;
    D=B*B-4*A*Q;
    if D<=0; hit=false; return; end
    t1=(-B-sqrt(D))/(2*A); t2=(-B+sqrt(D))/(2*A);
    hit = (min(t1,t2)<=1) && (max(t1,t2)>=0);
end

function refined = refineBinary(p,F0,env)
    refined = p; step=[pi/36,2.5,0.5,0.25];
    for r=1:3
        for idx=1:4
            s=step(idx)/r; phi=(1+sqrt(5))/2;

```

```

    cur = refined;
    for iter=1:12
        L=cur(idx)-s; R=cur(idx)+s;
        m1=R-(R-L)/phi; m2=L+(R-L)/phi;
        p1=cur; p2=cur; p1(idx)=m1; p2(idx)=m2;
        v1=highPrecisionEvaluate(p1,F0,env);
        v2=highPrecisionEvaluate(p2,F0,env);
        if v1>v2, cur(idx)=m1; else, cur(idx)=m2; end
        s=(R-L)/2; if s<1e-6, break; end
    end
    if
        highPrecisionEvaluate(cur,F0,env)>highPrecisionEvaluate(refined,F0,env)
            refined=cur;
        end
    end
    refined = jointOpt(refined,F0,env);
end

function val = highPrecisionEvaluate(p,F0,env)
    dt=0.01; th=p(1); v=p(2); t_rel=p(3); fuse=p(4);
    if v<70||v>140||t_rel<0||t_rel>30||fuse<0.1||fuse>8, val=0; return; end
    dir=[cos(th),sin(th),0]; P_drop=F0+v*t_rel*dir; C_e=P_drop+v*dir*fuse;
    C_e(3)=C_e(3)-0.5*env.g*fuse^2;
    t_e=t_rel+fuse; if C_e(3)<=0||t_e<0, val=0; return; end
    val=0;
    for t=t_e:dt:(t_e+env.T_eff)
        Mpos=env.M0+env.v_dandao*t*env.u_dandao; Ccloud=C_e-
[0,0,env.v_yanwu*(t-t_e)];
        if occludedHigh(Mpos,env.C_t,Ccloud,env.R_t,env.H_t,env.R_yanwu)
            val=val+dt;
        end
    end
end

function flag = occludedHigh(Mpos,Ct,Cc,Rt,Ht,Rs)
    Cb=[Ct(1),Ct(2),Ct(3)-Ht/2]; n=72;
    pts=sampleRim(Cb,Rt,Ht,n); flag=true;
    for k=1:size(pts,1)
        if ~segSphere(Mpos,pts(k,:),Cc,Rs), flag=false; return; end
    end
end

```

```

function best = jointOpt(p,F0,env)
    best=p; step=[pi/180,0.1,0.01,0.01];
    bestVal=highPrecisionEvaluate(p,F0,env);
    for it=1:50
        improve=false;
        for idx=1:4
            for dir=[-1,1]
                tmp=best; tmp(idx)=tmp(idx)+dir*step(idx);
                if idx==1, tmp(1)=mod(tmp(1),2*pi);
                elseif idx==2, if tmp(2)<70||tmp(2)>140, continue; end
                elseif idx==3, if tmp(3)<0||tmp(3)>30, continue; end
                elseif idx==4, if tmp(4)<0.1||tmp(4)>8, continue; end
                end
                v=highPrecisionEvaluate(tmp,F0,env);
                if v>bestVal, best=tmp; bestVal=v; improve=true; end
            end
        end
        if ~improve, step=step*0.8; if max(step)<1e-6, break; end; end
    end
end

```

问题五关键代码:

```

#长长长长! 2 小时以上
import numpy as np
from math import cos, sin, pi, sqrt, degrees
import random

np.random.seed(2025)
random.seed(2025)

class Shijiao:
    mubiao = np.array([0.0, 200.0, 0.0])
    cyl_r = 7.0
    cyl_h = 10.0
    yanwu_r = 10.0
    yanwu_v = 3.0
    yanwu_life = 20.0
    g = 9.8
    fyl_init = np.array([0.0,0.0,0.0])

```



```

daodan = np.array([
    [20000,0,2000],
    [19000,600,2100],
    [18000,-600,1900]
])
daodan_v = 300.0
fy1_start = np.array([6000.0,-3000.0,700.0])
dt = 0.01
refine = 10
t_max_det = 35.0
t_max_throw = 28.0
t_min_interval = 1.0

def unit_vector(v):
    n = np.linalg.norm(v); return v/n if n!=0 else v

def daodan_pos(t, m0):
    dir_vec = unit_vector(Shijiao.fy1_init - m0)
    return m0 + Shijiao.daodan_v * dir_vec * t

def fy1_pos(t, theta, v):
    dir_xy = np.array([cos(theta), sin(theta), 0.0])
    return Shijiao.fy1_start + v*dir_xy*t

def kaihuo(theta, v, t_tou, tau):
    r0 = fy1_pos(t_tou, theta, v)
    accel = np.array([0.0,0.0,-Shijiao.g])
    r_det = r0 + np.array([v*cos(theta),v*sin(theta),0.0])*tau +
0.5*accel*(tau**2)
    return r0, r_det, t_tou+tau

def yanwu_pos(t, r_det, t_det):
    if t < t_det or t - t_det > Shijiao.yanwu_life: return None
    dt = t - t_det
    return r_det + np.array([0.0,0.0,-Shijiao.yanwu_v*dt])

def cyl_pts():
    n_rad, n_h = 8, 2
    pts=[]
    zs = np.linspace(0,Shijiao.cyl_h,n_h)
    for z in zs:
        for i in range(n_rad):
            th = 2*pi*i/n_rad

```

```

        x = Shijiao.mubiao[0]+Shijiao.cyl_r*cos(th)
        y = Shijiao.mubiao[1]+Shijiao.cyl_r*sin(th)
        pts.append([x,y,z])
    pts.append([Shijiao.mubiao[0],Shijiao.mubiao[1],0])
    pts.append([Shijiao.mubiao[0],Shijiao.mubiao[1],Shijiao.cyl_h])
    return np.array(pts)

def seg_hits_sphere(p0,p1,c,R):
    d = p1-p0; f = p0-c
    a = np.dot(d,d); b=2*np.dot(f,d); c0=np.dot(f,f)-R**2
    disc = b*b-4*a*c0
    if disc<0: return False
    s1 = (-b - sqrt(max(0,disc)))/(2*a)
    s2 = (-b + sqrt(max(0,disc)))/(2*a)
    return 0<=s1<=1 or 0<=s2<=1

def merge_intervals(intervals):
    if len(intervals)==0: return np.zeros((0,2))
    intervals=sorted(intervals,key=lambda x:x[0])
    merged=[intervals[0]]
    for a,b in intervals[1:]:
        la,lb=merged[-1]
        if a<=lb+1e-9: merged[-1][1]=max(lb,b)
        else: merged.append([a,b])
    return np.array(merged)

def intersect_two(A,B):
    i=j=0; out=[]
    while i<len(A) and j<len(B):
        a1,b1=A[i]; a2,b2=B[j]
        lo,hi=max(a1,a2), min(b1,b2)
        if lo<hi: out.append([lo,hi])
        if b1<b2: i+=1
        else: j+=1
    return np.array(out)

def intersect_all(lst):
    out=lst[0]
    for u in lst[1:]:
        out=intersect_two(out,u)
        if len(out)==0: break
    return out

```

```

def total_len(U):
    return np.sum(U[:,1]-U[:,0]) if len(U)>0 else 0.0

def clamp_params(theta,v,trels,taus):
    theta=theta%(2*pi)
    v=np.clip(v,70,140)
    trels=np.clip(trels,0,Shijiao.t_max_throw); trels=np.sort(trels)
    for i in range(1,3):
        if trels[i]<trels[i-1]+Shijiao.t_min_interval:
            trels[i]=min(Shijiao.t_max_throw, trels[i-
1]+Shijiao.t_min_interval)
    taus=np.maximum(taus,0)
    for k in range(3):
        t_det=trels[k]+taus[k]
        if t_det>Shijiao.t_max_det:
            over=t_det-Shijiao.t_max_det
            dec=min(taus[k],over)
            taus[k]-=dec; over-=dec
            if over>1e-9: trels[k]=max(0,trels[k]-over)
    return theta,v,trels,taus

def intervals_point(theta,v,t_tou,tau,idx,m0):
    r0,r_det,t_det=kaihuo(theta,v,t_tou,tau)
    if not (0<=t_det<=Shijiao.t_max_det): return np.zeros((0,2))
    ts=np.arange(t_det,t_det+Shijiao.yanwu_life+1e-12,Shijiao.dt)
    pts=cyl_pts()
    def hit(t):
        c=yanwu_pos(t,r_det,t_det)
        return seg_hits_sphere(daodan_pos(t,m0),pts[idx],c,Shijiao.yanwu_r) if
c is not None else False
    out=[]; prev=hit(ts[0])
    if prev: out.append([ts[0],np.nan])
    for t in ts[1:]:
        s=hit(t)
        if s!=prev:
            lo,hi=ts[0],t
            for _ in range(Shijiao.refine): mid=0.5*(lo+hi); sm=hit(mid);
lo=mid if sm==prev else lo; hi=mid if sm!=prev else hi
            tsw=0.5*(lo+hi)
            if not prev and s: out.append([tsw,np.nan])
            elif prev and not s and len(out)>0 and np.isnan(out[-1][1]): out[-
1][1]=tsw
            prev=s

```

```

    if prev and len(out)>0 and np.isnan(out[-1][1]): out[-1][1]=ts[-1]
    out=[iv for iv in out if iv[1]-iv[0]>1e-9]
    return merge_intervals(out)

def bomb_for_missile(theta,v,t_tou,tau,m0):
    per=[intervals_point(theta,v,t_tou,tau,i,m0) for i in range(6)]
    return {'r_launch':kaihuo(theta,v,t_tou,tau)[0],
            'r_det':kaihuo(theta,v,t_tou,tau)[1],
            't_det':kaihuo(theta,v,t_tou,tau)[2],
            'per_point':per,
            'union':merge_intervals(np.vstack(per))}

def strict_union_3bombs(B1,B2,B3):
    unions=[merge_intervals(np.vstack([B1['per_point'][i],B2['per_point'][i],B
3['per_point'][i]])) for i in range(6)]
    cov=intersect_all(unions)
    return cov,total_len(cov)

# DE 优化
def rand_individual():
    theta=2*pi*np.random.rand(); v=70+70*np.random.rand()
    trels=np.sort(Shijiao.t_max_throw*np.random.rand(3))
    for i in [1,2]: trels[i]=max(trels[i],trels[i-1]+Shijiao.t_min_interval)
    taus=8*np.random.rand(3)
    return
np.hstack([theta,v,trels[0],taus[0],trels[1],taus[1],trels[2],taus[2]])

def objective(theta,v,trels,taus):
    theta,v,trels,taus=clamp_params(theta,v,np.array(trels),np.array(taus))
    total=0.0
    for m0 in Shijiao.daodan:
        bombs=[bomb_for_missile(theta,v,trels[k],taus[k],m0) for k in
range(3)]
        _,J=strict_union_3bombs(bombs[0],bombs[1],bombs[2])
        total+=J
    return total

def eval_x(x): return objective(x[0],x[1],[x[2],x[4],x[6]],[x[3],x[5],x[7]])

def run_de():
    pop,gens,F,CR=56,250,0.7,0.9
    X=np.array([rand_individual() for _ in range(pop)])
    fit=np.array([eval_x(X[i,:]) for i in range(pop)])

```

```

    best_idx=np.argmax(fit); best_fit=fit[best_idx]; best=X[best_idx,:]
    print("[DE 开始][三导弹] 粗解计算中...")
    for g in range(gens):
        for i in range(pop):
            idxs=[j for j in range(pop) if j!=i];
a,b,c=np.random.choice(idxs,3,replace=False)
            mutant=X[a]+F*(X[b]-X[c])+0.2*(best-X[a])
            cross=np.array([mutant[j] if np.random.rand()<CR or
j==np.random.randint(len(mutant)) else X[i,j] for j in range(len(mutant))])
            cross[0]=cross[0]%(2*pi); cross[1]=np.clip(cross[1],70,140)
            for idx in [2,4,6]:
cross[idx]=np.clip(cross[idx],0,Shijiao.t_max_throw)
            for idx in [3,5,7]: cross[idx]=np.clip(cross[idx],0,8)
            val=eval_x(cross)
            if val>=fit[i]: X[i,:]=cross; fit[i]=val;
            if val>best_fit: best_fit=val; best=cross
                print(f"[DE][三导弹] gen={g+1}/{gens}, best J_total={best_fit:.6f}")
    print(f"\n[DE 结束][三导弹] 粗解 J_total={best_fit:.6f}")
    return best

if __name__=="__main__":
    best=run_de()
    print("优化完成, 最佳向量: ", best)

```

```

clear; clc;
%快, 但主要用 PSO 吧
% 固定参数
g = 9.81; v_dandao = 300; v_yanwu = 3; R_yanwu = 10; T_eff = 20;
R_t = 7; H_t = 10;
C_t = [0, 200, 5]; M0 = [20000, 0, 2000];
F0 = [12000, 1400, 1400];
u_dandao = -M0 / norm(M0);
dt = 0.01; nTheta = 720;

% 边界
lb = [0, 70, 0, 0.1];
ub = [2*pi, 140, 30, 8.0];

% SA 参数
SA.MaxStages = 180; SA.L_per_stage = 220; SA.alpha = 0.93;
SA.reheat_every = 25; SA.reheat_mult = 1.4;

```

```

SA.target_acc_hi = 0.60; SA.target_acc_lo = 0.20;
SA.step_up = 1.2; SA.step_dn = 0.8; SA.Tmin = 1e-4;
rng(42);

rimPts = buildRimPoints(C_t, R_t, H_t, nTheta);
objFun = @(x) objectiveFunction(x, M0, F0, C_t, u_dandao, g, v_dandao,
v_yanwu, ...
                                R_yanwu, R_t, H_t, T_eff, dt, rimPts);

x0 = lb + (ub - lb).*rand(1,4);
DeltaS = etd(x0, 30, lb, ub, objFun);
T0 = max(1e-6, -DeltaS / log(0.8));
sigma0 = 0.10*(ub - lb); sigma = sigma0;
sigma_min = 1e-4*(ub - lb); sigma_max = 0.25*(ub - lb);

f0 = objFun(x0); x = x0; fx = f0; best_solution = x; f_best = fx;

T = T0; no_improve_stages = 0;
for stage = 1:SA.MaxStages
    n_accept = 0;
    for l = 1:SA.L_per_stage
        x_new = neighbor_gauss_reflect(x, sigma, lb, ub);
        f_new = objFun(x_new);
        dF = f_new - fx;
        accept = (dF <= 0) || (rand < exp(-dF / T));
        if accept
            x = x_new; fx = f_new; n_accept = n_accept + 1;
            if fx < f_best
                best_solution = x; f_best = fx;
            end
        end
    end
    acc_rate = n_accept / SA.L_per_stage;
    if acc_rate > SA.target_acc_hi
        sigma = min(sigma_max, SA.step_up * sigma);
    elseif acc_rate < SA.target_acc_lo
        sigma = max(sigma_min, SA.step_dn * sigma);
    end
    T = max(SA.Tmin, SA.alpha * T);
    if n_accept == 0
        no_improve_stages = no_improve_stages + 1;
        if no_improve_stages >= SA.reheat_every
            T = min(T0, SA.reheat_mult * T); no_improve_stages = 0;

```

```

        end
    else
        no_improve_stages = 0;
    end
    if T <= SA.Tmin, break; end
end

function x_new = neighbor_gauss_reflect(x, sigma, lb, ub)
    x_new = x + sigma .* randn(size(x));
    for i = 1:numel(x_new)
        while x_new(i) < lb(i) || x_new(i) > ub(i)
            if x_new(i) < lb(i)
                x_new(i) = lb(i) + (lb(i) - x_new(i));
            end
            if x_new(i) > ub(i)
                x_new(i) = ub(i) - (x_new(i) - ub(i));
            end
        end
    end
end

function DeltaS = etd(x_ref, N, lb, ub, fhandle)
    f0 = fhandle(x_ref);
    deltas = zeros(N,1);
    for k = 1:N
        x_try = lb + (ub - lb).*rand(1, numel(x_ref));
        fk = fhandle(x_try);
        deltas(k) = abs(fk - f0);
    end
    DeltaS = median(deltas);
    if ~isfinite(DeltaS) || DeltaS <= 0, DeltaS = 1; end
end

function hit = lineSphereIntersect(M, p, c, r)
    d = p - M; f = M - c; A = dot(d, d);
    if A <= 1e-16, hit = (dot(f,f) <= r*r); return; end
    B = 2*dot(f,d); Q = dot(f,f)-r^2; D = B^2 - 4*A*Q;
    if D <= 0, hit = false; return; end
    t1 = (-B - sqrt(D)) / (2*A); t2 = (-B + sqrt(D)) / (2*A);
    hit = (max(t1,t2) >= 0) && (min(t1,t2) <= 1);
end

function rimPts = buildRimPoints(C_t, R_t, H_t, nTheta)

```

```

C_base = [C_t(1), C_t(2), C_t(3) - H_t/2];
thetas = linspace(0, 2*pi, nTheta+1); thetas(end) = [];
rimPts = zeros(2*nTheta, 3);
for i = 1:nTheta
    c = cos(thetas(i)); s = sin(thetas(i));
    rimPts(2*i-1,:) = [C_base(1)+R_t*c, C_base(2)+R_t*s, C_base(3)];
    rimPts(2*i,:) = [C_base(1)+R_t*c, C_base(2)+R_t*s, C_base(3)+H_t];
end
end

[~, P_drop, C_e, t_e] = objectiveFunction(best_solution, M0, F0, C_t,
u_dandao, g, v_dandao, v_yanwu, ...
R_yanwu, R_t, H_t, T_eff, dt, rimPts);
fprintf('最优参数: theta=%.2f°, v=%.2f, t_rel=%.2f, t_fuse=%.2f\n', ...
best_solution(1)*180/pi, best_solution(2), best_solution(3),
best_solution(4));
fprintf('最大完全遮蔽时长: %.3f 秒\n', -f_best);

function [fitness, P_drop, C_e, t_e] = objectiveFunction(x, M0, F0, C_t,
u_dandao, g, v_dandao, v_yanwu, ...
R_yanwu, R_t, H_t,
T_eff, dt, rimPts)
    theta1 = x(1); v_u = x(2); t_rel = x(3); t_fuse = x(4);
    dir = [cos(theta1), sin(theta1), 0];
    P_drop = F0 + v_u * t_rel * dir;
    v_drop = v_u * dir;
    C_e = P_drop + v_drop * t_fuse; C_e(3) = C_e(3) - 0.5 * g * t_fuse^2;
    t_e = t_rel + t_fuse;
    if C_e(3) <= 0 || t_e < 0, fitness = 1e3; return; end
    T_cover = 0;
    timeVec = t_e:dt:t_e+T_eff;
    coveredTime = sum(arrayfun(@(t) ...
        irs(M0 + v_dandao*t*u_dandao, C_e - [0,0,v_yanwu*(t-t_e)], R_yanwu,
rimPts), ...
        timeVec)) * dt;
    fitness = -T_cover;
end

function occluded = irs(M_pos, C_cloud, R_s, rimPts)
    occluded = true;
    for k = 1:size(rimPts,1)
        if ~lineSphereIntersect(M_pos, rimPts(k,:), C_cloud, R_s)
            occluded = false; return;
        end
    end
end

```



```
        end
    end
end
```